Accretion

SECURITY ASSESSMENT

Provided by Accretion Labs Pte Ltd. for Metadao
March 06, 2026
A26MET1

Metadao

AUDITORS

Role	Name
Lead Auditor	Robert Reith (robert@accretion.xyz)
Auditor	Mahdi Rostami (mahdi@accretion.xyz)
Auditor	J4x (contact@accretion.xyz)

CLIENT

Metadao (https://metadao.fi) A comprehensive security audit of the MetaDAO protocol's smart contract infrastructure and governance mechanisms.

ENGAGEMENT TIMELINE

17 Jan — **Review Fixes**
Security recommendations are given and implemented

18 Jan — **Project Kickoff**
Initial planning and scope definition

18 Jan — **Assessment Begins**
Security review and testing phase

23 Jan — **Project Completion**
Report delivery and on-chain confirmation

AUDITED CODE

#	Program ID	Repository
1	FUTARELBfJfQ8RDGhg1wdhddq1odMAJUePHFuBYfUxKq	https://github.com/metaDAOproject/programs
2	moontUzsdepotRGe5xsfip7vLPTJnVuafqdUWexVnPM	https://github.com/metaDAOproject/programs
3	WALL8ucBuUyL46QYxwYJjidaFYhdvxUFrgvBxPshERx	https://github.com/metaDAOproject/programs
4	pbPPQH7jyKoSLu8QYs3rSY3YkDRXEBojkbTgnUg7NDS	https://github.com/metaDAOproject/programs
5	VLTX1ishMBbcX3rdBWGssxawAo1Q2X2qxYFYqiGodVg	https://github.com/metaDAOproject/programs

ASSESSMENT

The security assessment of metadao's programs revealed 47 total findings across multiple severity levels: 1 Critical, 14 Medium, 20 Low, and 12 Informational issues. The development team has demonstrated strong responsiveness to security concerns, successfully addressing all relevant security issues.

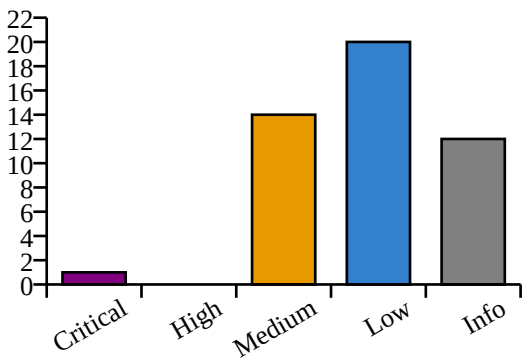
CODE ASSESSMENT

The codebase is built on a multi-program architecture with clear separation of concerns and thorough testing implementing both a full futarchy system and a launchpad. The code shows good use of Anchor framework conventions and includes comprehensive validation logic, though some areas revealed gaps in input validation and edge case handling.

KEY FINDINGS

The most critical finding involved integer truncation in liquidity withdrawal operations that could lead to loss of funds, which was promptly fixed. Medium severity issues primarily centered around economic vulnerabilities including fee misdirection, frontrunning opportunities in token sales, and governance bypasses through proposal staking mechanisms.

SEVERITY DISTRIBUTION



© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ENGAGEMENT SCOPE

The scope of this security assessment was a full review of the following items:

Item 1: futarchy
Link: https://github.com/metaDAOproject/programs
Commit: d05c432f9b67221267b0b3fd009cfd0227b4605b
Program ID: FUTARELBfJfQ8RDGhg1wdhddq1odMAJUePHFuBYfUxKq
Audit Result:
- **Audited Commit:** 25abdade0a3be1e2243a54ccfa8e15e8dbf4c5cb
- **Build Hash:** 374a86d375d473ac307a5dd83f063bbb3a39608ae469a083133993081e963129
- **Status:** verified

Item 2: launchpad
Link: https://github.com/metaDAOproject/programs
Commit: d05c432f9b67221267b0b3fd009cfd0227b4605b
Program ID: moontUzsdepotRGe5xsfip7vLPTJnVuafqdUWexVnPM
Audit Result:
- **Audited Commit:** 25abdade0a3be1e2243a54ccfa8e15e8dbf4c5cb
- **Build Hash:** f6a7a12734ba568bb7afac2d46808fc0303c00bede0ca4beb1d7e75e8fe8af40
- **Status:** verified

Item 3: bid wall
Link: https://github.com/metaDAOproject/programs
Commit: d05c432f9b67221267b0b3fd009cfd0227b4605b
Program ID: WALL8ucBuUyL46QYxwYJjidaFYhdvxUFrgvBxPshERx
Audit Result:
- **Audited Commit:** 25abdade0a3be1e2243a54ccfa8e15e8dbf4c5cb
- **Build Hash:** 4ae44aee1255f270a15f9af6b27b76512f50bdf8e04afabba508a30666dd1ae5
- **Status:** verified

Item 4: price based performance package
Link: https://github.com/metaDAOproject/programs
Commit: d05c432f9b67221267b0b3fd009cfd0227b4605b
Program ID: pbPPQH7jyKoSLu8QYs3rSY3YkDRXEBojKbTgnUg7NDS
Audit Result:
- **Audited Commit:** 3a0da4e9eaad52a2f48b164c2d40334f86622a51
- **Build Hash:** 14c44d4e8d8ae41567b2127f18061001fe41ab6422a9ea33fdbea00b756bc93b
- **Status:** verified

Item 5: Conditional Vault
Link: https://github.com/metaDAOproject/programs
Commit: d05c432f9b67221267b0b3fd009cfd0227b4605b
Program ID: VLTX1ishMBbcX3rdBWGssxawAo1Q2X2qxYFYqiGodVg
Audit Result:
- **Audited Commit:** 25abdade0a3be1e2243a54ccfa8e15e8dbf4c5cb
- **Build Hash:** 93585d0c6302f8af7b0f2ced13527dad0242ac0cbbdeb8d8591857d0b1292cfc
- **Status:** Verified

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ISSUES SUMMARY

ID	TITLE	SEVERITY	STATUS
ACC-C1	Integer Truncation Leads to LoF on Liquidity Withdraw	CRITICAL	FIXED
ACC-M1	Multisig Config Changes Ignore Optimistic Proposals and Can Freeze Dao	MEDIUM	FIXED
ACC-M2	Recreating Spending Limits Can Remove DAO Create Key and Break Governance Spend Flow	MEDIUM	ACKNOWLEDGED
ACC-M3	Duplicate Multisig Members Can Permanently Block Launchpad Completion	MEDIUM	FIXED
ACC-M4	`sell_tokens` Rounding Causes NAV Drift and Decreasing Effective Price	MEDIUM	FIXED
ACC-M5	Closing PerformancePackage Leaves Dangling Change Requests	MEDIUM	ACKNOWLEDGED
ACC-M6	AdminApproveExecuteMultisigProposal Will Not Work In Production Due To Reentrancy	MEDIUM	FIXED
ACC-M7	`split_tokens` and `merge_tokens` Do Not Verify Question Is Unresolved	MEDIUM	FIXED
ACC-M8	Oracle Config Change After Unlock Start Corrupts TWAP Calculation	MEDIUM	FIXED
ACC-M9	Launch Can Be Front-Run When Base Mint Is Created Separately	MEDIUM	ACKNOWLEDGED
ACC-M10	`initialize_proposal` Stale Check Can Be Bypassed with Unrelated Multisig	MEDIUM	FIXED
ACC-M11	LP Position Hijack Can Lock Liquidity Until Recovery	MEDIUM	FIXED
ACC-M12	Proposal staking minimum can be bypassed	MEDIUM	ACKNOWLEDGED
ACC-M13	`sell_tokens` Can Be Frontrun to Force Worse Price	MEDIUM	FIXED
ACC-M14	Fees sent to wrong recipient in case of `collect_fees` instruction	MEDIUM	FIXED
ACC-L1	Pricebased Performance Package Propose Change Lacks Correct Event Handling	LOW	FIXED
ACC-L2	`complete_launch` Creates and Funds Treasury Quote Account Even in Refunding State	LOW	ACKNOWLEDGED
ACC-L3	Complete Unlock Resets to Locked When No Tranche Unlocks, Enabling Griefing	LOW	UNKNOWN
ACC-L4	Launch Can Be Funded and Closed in the Same Second at Period End	LOW	FIXED
ACC-L5	Bid Wall Price Can Be Inflated by Donating to DAO Treasury	LOW	WONTFIX

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L6	Unnecessary Enforcement of ATA	LOW	ACKNOWLEDGED
ACC-L7	Performance Package Token Vault Not Closed After Burn	LOW	ACKNOWLEDGED
ACC-L8	Unused `unlocked` State Causes Incorrect Performance Package Finalization	LOW	FIXED
ACC-L9	Proposal Can Be Traded After Expiry Before Finalization	LOW	UNKNOWN
ACC-L10	DAO parameters can be modified during active futarchy markets, undermining trader assumptions	LOW	FIXED
ACC-L11	Premine Cap Check Ignores `additional_tokens_amount`	LOW	FIXED
ACC-L12	MAX_PREMINE Allows More Than 50% of Total Supply	LOW	FIXED
ACC-L13	Performance Package Over-allocates Rent for Unused Tranches	LOW	ACKNOWLEDGED
ACC-L14	AMM Position Rent Can Never Be Reclaimed	LOW	ACKNOWLEDGED
ACC-L15	`provide_liquidity` overcharges base amount by 1 unit in case of no rounding	LOW	FIXED
ACC-L16	`provide_liquidity` Ignores `min_liquidity` slippage parameter for First LP	LOW	FIXED
ACC-L17	Stake Account Rent Is Never Reclaimed	LOW	ACKNOWLEDGED
ACC-L18	Zero Address `position_authority` Can Permanently Lock Funds	LOW	WONTFIX
ACC-L19	`sell_tokens` Can Result in Zero Output Due to Rounding	LOW	FIXED
ACC-L20	`close_bid_wall` is missing authority signature allowing for rent theft	LOW	FIXED
ACC-I1	Integer Overflow in Total Token Amount Calculation During Performance Package Initialization	INFO	FIXED
ACC-I2	Missing Event Emission	INFO	FIXED
ACC-I3	Refund Requires User ATA	INFO	FIXED
ACC-I4	`resize_dao` Uses DAO Balance for Rent Calculation and Can Misbehave if DAO Holds SOL	INFO	UNKNOWN
ACC-I5	Performance Package Init Accepts Default Pubkey for Oracle, Grantee, and Authority	INFO	UNKNOWN
ACC-I6	When a Proposal is Rejected by the Market The Squads Proposal Should Be Rejected	INFO	FIXED
ACC-I7	Invariants in DAO State Can Be Improved	INFO	ACKNOWLEDGED
ACC-I8	Misleading Variable Name `input_amount_with_lp_fee` in Futarchy AMM Swap	INFO	FIXED

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I9	Outdated "Autocrat" References Should Be Replaced With "Futarchy"	INFO	FIXED
ACC-I10	Incorrect Reference Link in Sqrt Price Calculation	INFO	FIXED
ACC-I11	Duplicate Optimistic Proposal Check in `initiate_vault_spend_optimistic_proposal`	INFO	FIXED
ACC-I12	`unreachable!()` in AMM Operations leads to no error returns	INFO	FIXED

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

DETAILED ISSUES

ACC-C1 Integer Truncation Leads to LoF on Liquidity Withdraw

CRITICAL FIXED

Description

We found that when calculating the withdrawal amounts, the `WithdrawLiquidity` instruction downcasts `liquidity_to_withdraw` and `total_liquidity` from u128 to u64 unsafely. By engineering the `liquidity_to_withdraw` parameter in such a way that the downcasted amount is in larger proportion to the downcasted `total_liquidity` than their non-downcasted versions, which is only possible when `total_liquidity` is larger than u64::MAX, we can effectively receiver more tokens than we are entitled to, allowing us to potentially drain the protocol.

Location

https://github.com/metaDAOproject/programs/blob/d05c432f9b67221267b0b3fd009cfd0227b4605b/programs/futarchy/src/instructions/withdraw_liquidity.rs#L99-L102

Relevant Code

```
/// withdraw_liquidity.rs L99-L102
            spot.get_base_and_quote_withdrawable(
                liquidity_to_withdraw as u64,
                total_liquidity as u64,
            )
```

Mitigation Suggestion

Refactor the `get_base_and_quote_withdrawable` function to work with u128 integers so that downcasts aren't necessary.

Remediation

Fixed in commit `1fe350a9b410acb450608247ed2443b2608a3ede`.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M1 Multisig Config Changes Ignore Optimistic Proposals and Can Freeze Dao

MEDIUM FIXED

Description

Once an optimistic proposal passes its threshold time, no new proposals can be created until it is finalised. If finalisation fails, governance can get stuck with no way to launch new proposals.

Finalising an optimistic proposal CPI-calls Squads to approve the linked transaction. Squads can reject approval if the transaction becomes **stale**. A multisig config change makes **all transactions up to that moment stale**, not only ones before the config transaction.

The only way to change multisig config is `AdminApproveExecuteMultisigProposal`. This function checks for active proposals, but the check **does not include optimistic proposals**. So an admin can change config while an optimistic proposal is waiting for finalisation, making its transaction stale and blocking finalisation. Since new proposals cannot be created until the optimistic one is finalised, governance can become stuck.

If we check initialize_proposal.rs link, we will see that if the proposal becomes stale, there is no way to initialse proposal and clean opmitisic propsal so, so that means from the moment we initiate an optimistic proposal, if the config changes, dao will freeze.

Location

https://github.com/metaDAOproject/programs/blob/d05c432f9b67221267b0b3fd009cfd0227b4605b/programs/futarchy/src/instructions/admin_approve_execute_multisig_proposal.rs#L63-L65

Relevant Code

```
if !matches!(self.dao.amm.state, PoolState::Spot { .. }) {
            return Err(FutarchyError::PoolNotInSpotState.into());
        }
```

Mitigation Suggestion

Check for active optimistic proposals as well.

```
        // There should be no active optimistic proposal
        require!(
            self.dao.optimistic_proposal.is_none(),
            FutarchyError::ActiveOptimisticProposalAlreadyEnqueued
        );
```

Remediation

Fixed in https://github.com/metaDAOproject/programs/commit/cf0faccc32cb552cd06ba96b44d35f05beb66f41.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M2 Recreating Spending Limits Can Remove DAO Create Key and Break Governance Spend Flow

MEDIUM ACKNOWLEDGED

Description

We found that users can delete a spending limit and create a new one. If they delete the default spending limit (the one created with the DAO as `create_key`), there is no guarantee the replacement spending limit will also use the DAO as `create_key`.

If a new spending limit is created with a different `create_key`, optimistic governance spending can break because downstream logic relies on the DAO-created spending limit.

Location

https://github.com/metaDAOproject/programs/blob/d05c432f9b67221267b0b3fd009cfd0227b4605b/programs/futarchy/src/instructions/initiate_vault_spend_optimistic_proposal.rs#L18-L19

Relevant Code

```
#[account(seeds = [squads_multisig_program::SEED_PREFIX, squads_multisig.key().as_ref(), squads_multisig_program::SEED_SPENDING_LIMIT, dao.key().as_ref()], bump, seeds::program = squads_program)]
    pub squads_spending_limit: Account<'info, squads_multisig_program::SpendingLimit>,
```

Mitigation Suggestion

-

Remediation

Acknowledged.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M3 Duplicate Multisig Members Can Permanently Block Launchpad Completion

MEDIUM FIXED

Description

Launchpad completion after a certain time is permissionless and should never be blockable. The launchpad must always be able to move into a final state (refund or complete), otherwise user funds can get stuck in the contract.

We found that the completion flow depends on creating a Squads spending limit, and Squads rejects duplicate members in `monthly_spending_limit_members` (duplicate check here). Link

If the launchpad is initialized with duplicate members, completion will always fail due to this validation, permanently blocking finalization and potentially locking user funds.

Location

https://github.com/metaDAOproject/programs/blob/d05c432f9b67221267b0b3fd009cfd0227b4605b/programs/v07_launchpad/src/instructions/initialize_launch.rs#L24

https://github.com/metaDAOproject/programs/blob/d05c432f9b67221267b0b3fd009cfd0227b4605b/programs/v07_launchpad/src/instructions/complete_launch.rs#L435

https://github.com/Squads-Protocol/v4/blob/5982942f81d3ad1cc9f3c431cd5d73da5f4cfc57/programs/squads_multisig_program/src/state/spending_limit.rs#L74-L76

Relevant Code

```
// There must be no duplicate members, we make sure members are sorted when creating a SpendingLimit.
        let has_duplicates = self.members.windows(2).any(|win| win[0] == win[1]);
        require!(!has_duplicates, MultisigError::DuplicateMember);
```

Mitigation Suggestion

Validate and deduplicate `monthly_spending_limit_members` during launchpad initialization so completion can never fail due to duplicate members.

Remediation

Fixed in https://github.com/metaDAOproject/programs/commit/6e8c5aa32e5a3ef39384ea72fc84e0014ae3f92d.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M4 `sell_tokens` Rounding Causes NAV Drift and Decreasing Effective Price

MEDIUM FIXED

Description

The `sell_tokens` instruction is intended to let users sell base tokens at the launch-adjusted price (total_nav / remaining_base), accounting for treasury changes. NAV and the denominator are computed as:

```
let total_nav: u128 = (ctx.accounts.bid_wall.initial_amm_quote_reserves
    + ctx.accounts.dao_treasury_quote_token_account.amount
    + ctx.accounts.bid_wall.quote_amount) as u128;

let remaining_base =
    (TOKENS_TO_PARTICIPANTS - ctx.accounts.bid_wall.base_bought_amount) as u128;
```

Payout is then computed with integer division and the wall's `quote_amount` is debited by that same value. Because the division rounds down, the wall is reduced by less than the exact proportional amount:

```
let amount_out_before_fee =
    (amount_in as u128 * total_nav as u128 / remaining_base as u128) as u64;
// ...
ctx.accounts.bid_wall.quote_amount -= amount_out_before_fee;
```

`base_bought_amount` (and hence `remaining_base`) is updated exactly, so the denominator has no rounding:

```
ctx.accounts.bid_wall.base_bought_amount += amount_in;
```

After each rounding sell, extra quote stays in the wall; that excess is part of `total_nav` on the next sell while the denominator stays exact. NAV therefore drifts above the intended invariant, and the effective price (quote per base received) is lower than intended and degrades with every sell that rounds.

Location

/programs/bid_wall/src/instructions/sell_tokens.rs#L94-L157

Relevant Code

```
// total_nav includes bid_wall.quote_amount; remaining_base is exact
let total_nav: u128 = (... + ctx.accounts.bid_wall.quote_amount) as u128;
let remaining_base = (TOKENS_TO_PARTICIPANTS - ctx.accounts.bid_wall.base_bought_amount) as u128;

// rounded-down payout and debit; exact base update
let amount_out_before_fee = (amount_in as u128 * total_nav / remaining_base) as u64;
ctx.accounts.bid_wall.quote_amount -= amount_out_before_fee;
ctx.accounts.bid_wall.base_bought_amount += amount_in;
```

Mitigation Suggestion

We recommend debiting `quote_amount` by a rounded-up value so the price never gets below the original one.

Remediation

Fixed in https://github.com/metaDAOproject/programs/commit/9d35ae8418bdd4803aa8367e33cfa98d1a3c390a

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

React

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M5 Closing PerformancePackage Leaves Dangling Change Requests

MEDIUM ACKNOWLEDGED

Description

We found that after closing a `PerformancePackage`, related **change requests** can remain dangling and cannot be closed. This has two problems: 1. Rent couldn't be claimed 2. There is **no nonce** in `PerformancePackage` creation and only **one package per creator** is allowed, so if the same creator recreates a package, old change requests may conflict with the new one or cause unexpected behaviour.

Location

https://github.com/metaDAOproject/programs/blob/d05c432f9b67221267b0b3fd009cfd0227b4605b/programs/price_based_performance_package/src/instructions/execute_change.rs#L16-L17

https://github.com/metaDAOproject/programs/blob/d05c432f9b67221267b0b3fd009cfd0227b4605b/programs/price_based_performance_package/src/instructions/initialize_performance_package.rs#L24

Relevant Code

```
    #[account(mut)]
    pub performance_package: Account<'info, PerformancePackage>,
```

```
        seeds = [b"performance_package", create_key.key().as_ref()],
```

Mitigation Suggestion

• Add a nonce/version to `PerformancePackage` or ensure all related change requests are closed and cleaned up before allowing package closure or recreation.

• Allow closing the Change request when there is no PerformancePackage.

Remediation

Acknowledged.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M6 AdminApproveExecuteMultisigProposal Will Not Work In Production Due To Reentrancy

MEDIUM FIXED

Description

We found that the `AdminApproveExecuteMultisigProposal` instruction will not work as intended in production. This instruction implements a backdoor which allows only an admin, which is to be the MetaDAO multisig, to call this instruction. The instruction shall then vote approve on any other DAO proposal and execute it. However, while this works in testing when the admin is set to a keypair, this will not work in production when the admin is a squads multisig itself. This is because the futarchy instruction will be called through the squads program, and then it will try to call the squads program again creating a `SQUADS -> FUTARCHY -> SQUADS` call chain which is a reentrancy into Squads which is forbidden by the Solana runtime.

Location

https://github.com/metaDAOproject/programs/blob/d05c432f9b67221267b0b3fd009cfd0227b4605b/programs/futarchy/src/instructions/admin_approve_execute_multisig_proposal.rs#L3-L16

Relevant Code

```
/// admin_approve_execute_multisig_proposal.rs L3-L16
pub mod metadao_multisig_vault {
    use anchor_lang::prelude::declare_id;

    // MetaDAO multisig
    declare_id!("6awyHMshBGVjJ3ozdSJdyyDE1CTAXUwrpNMaRGMsb4sf");
}

#[derive(Accounts)]
#[event_cpi]
pub struct AdminApproveExecuteMultisigProposal<'info> {
    #[account(mut, has_one = squads_multisig)]
    pub dao: Account<'info, Dao>,
    #[account(mut)]
    pub admin: Signer<'info>,
```

Mitigation Suggestion

The only way to make this work is if the admin is not a squads v4 DAO. For this, a squads v3 DAO could be set up, or a fork of squads v4 could be deployed, or a keypair would need to be used. Alternatively, this could be made a 2 step instruction, which splits the approval from the admin DAO and the execution into two separate instructions, bypassing the runtime restriction.

Remediation

Fixed in commit `450179d7d2048a64fa63c8037042c6000fcc0cbe` by using an admin keypair. A 2 step implementation will also be considered

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M7 `split_tokens` and `merge_tokens` Do Not Verify Question Is Unresolved

MEDIUM FIXED

Description

The `split_tokens` and `merge_tokens` instructions do not check whether the question has already been resolved. Unlike `redeem_tokens`, which uses `#[access_control(ctx.accounts.validate_redeem_tokens())]` and requires the question to be resolved, split, and merge, these methods have no access control that enforces that the question remains unresolved. Allowing split or merge after resolution is inconsistent with the intended lifecycle: once a question is resolved, conditional tokens should only be redeemed (via `redeem_tokens`), not split from or merged back to the underlying.

```
// lib.rs - split_tokens and merge_tokens have no access_control
pub fn split_tokens<'c: 'info, 'info>(
    ctx: Context<'_, '_, 'c, 'info, InteractWithVault<'info>>,
    amount: u64,
) -> Result<()> {
    InteractWithVault::handle_split_tokens(ctx, amount)
}

pub fn merge_tokens<'c: 'info, 'info>(
    ctx: Context<'_, '_, 'c, 'info, InteractWithVault<'info>>,
    amount: u64,
) -> Result<()> {
    InteractWithVault::handle_merge_tokens(ctx, amount)
}

// redeem_tokens correctly gates on resolution
#[access_control(ctx.accounts.validate_redeem_tokens())]
pub fn redeem_tokens<'c: 'info, 'info>(
    ctx: Context<'_, '_, 'c, 'info, InteractWithVault<'info>>,
) -> Result<()> {
    InteractWithVault::handle_redeem_tokens(ctx)
}
```

After resolution, conditional tokens have payoff semantics; splitting would mint new conditional tokens after the outcome is known, and merging would treat all outcome tokens as 1:1 with the underlying, which is incorrect. Both should be disallowed when the question is resolved.

Location

/programs/conditional_vault/src/lib.rs#L61-L74 /programs/conditional_vault/src/instructions/common.rs /programs/conditional_vault/src/instructions/split_tokens.rs /programs/conditional_vault/src/state/question.rs#L40-L42

Relevant Code

```
// redeem_tokens.rs - has validate_redeem_tokens requiring resolved
pub fn validate_redeem_tokens(&self) -> Result<()> {
    require!(
        self.question.is_resolved(),
        VaultError::CantRedeemConditionalTokens
    );
    Ok(())
}

// common.rs - InteractWithVault has no validate_split_tokens / validate_merge_tokens
pub struct InteractWithVault<'info> {
    pub question: Account<'info, Question>,
    #[account(mut, has_one = question)]
    pub vault: Account<'info, ConditionalVault>,
    // ...
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
}

// question.rs - is_resolved available
pub fn is_resolved(&self) -> bool {
    self.payout_denominator != 0
}
```

Mitigation Suggestion

We recommend adding validators that require the question is not resolved, and wiring them with `#[access_control]` for both instructions.

1. In `InteractWithVault` (e.g. in `common.rs`), add:

```
pub fn validate_split_tokens(&self) -> Result<()> {
    require!(
        !self.question.is_resolved(),
        VaultError::QuestionAlreadyResolved
    );
    Ok(())
}

pub fn validate_merge_tokens(&self) -> Result<()> {
    require!(
        !self.question.is_resolved(),
        VaultError::QuestionAlreadyResolved
    );
    Ok(())
}
```

2. In `lib.rs`, add access control for both entrypoints:

```
#[access_control(ctx.accounts.validate_split_tokens())]
pub fn split_tokens<'c: 'info, 'info>(
    ctx: Context<'_, '_, 'c, 'info, InteractWithVault<'info>>,
    amount: u64,
) -> Result<()> {
    InteractWithVault::handle_split_tokens(ctx, amount)
}

#[access_control(ctx.accounts.validate_merge_tokens())]
pub fn merge_tokens<'c: 'info, 'info>(
    ctx: Context<'_, '_, 'c, 'info, InteractWithVault<'info>>,
    amount: u64,
) -> Result<()> {
    InteractWithVault::handle_merge_tokens(ctx, amount)
}
```

This ensures split and merge are only allowed while the question is unresolved; after resolution, only redeem is valid.

Remediation

Fixed in https://github.com/metaDAOproject/programs/commit/3865e80491c3a2005e3515b4d3bd58551e6b91a3

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M8 Oracle Config Change After Unlock Start Corrupts TWAP Calculation

MEDIUM FIXED

Description

`propose_change` and `execute_change` do not check whether the performance package is already in `Unlocking` state. They allow changing `oracle_config` (e.g. switching to a different oracle account or byte layout). When unlock has already started, `start_aggregator` and `start_timestamp` in `performance_package.state` were captured from the previous oracle in `start_unlock`. If an oracle change is then applied, `complete_unlock` will read the current aggregator from the new oracle but still use the stored `start_aggregator` and `start_timestamp` from the old oracle, producing an incorrect or meaningless TWAP and wrong unlock behavior.

```
// execute_change.rs - no guard on performance_package.state
match &change_request.change_type {
    ChangeType::Oracle { new_oracle_config } => {
        performance_package.oracle_config = *new_oracle_config;
    }
    // ...
}
```

```
// complete_unlock.rs - uses stored start_* (from old oracle) with current oracle
let (start_aggregator, start_timestamp) = match &performance_package.state {
    PerformancePackageState::Unlocking {
        start_aggregator,
        start_timestamp,
    } => (*start_aggregator, *start_timestamp),
    _ => unreachable!(),
};

// Reads from performance_package.oracle_config (may now be the new oracle)
let current_aggregator =
    u128::from_le_bytes(oracle_data[offset..offset + 16].try_into().unwrap());
// ...
let aggregator_change = current_aggregator.saturating_sub(start_aggregator);
let twap_price = aggregator_change / time_passed as u128;
```

Attack flow: recipient (or authority) starts unlock → state becomes `Unlocking` with old oracle's baseline → a change request switches `oracle_config` to a new oracle → executor runs `execute_change` → `complete_unlock` runs later and mixes old baseline with new oracle's current value, corrupting the TWAP and thus which tranches unlock and how many tokens are sent.We found...

Location

/programs/price_based_performance_package/src/instructions/propose_change.rs
/programs/price_based_performance_package/src/instructions/execute_change.rs#L51-L59
/programs/price_based_performance_package/src/instructions/start_unlock.rs#L81-L84
/programs/price_based_performance_package/src/instructions/complete_unlock.rs#L82-L127

Relevant Code

```
// propose_change.rs - validate() does not check performance_package.state
pub fn validate(&self) -> Result<()> {
    if self.proposer.key() != self.performance_package.recipient
        && self.proposer.key() != self.performance_package.performance_package_authority
    {
        // ...
    }
    Ok(())
}
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
// execute_change.rs - applies oracle change regardless of Unlocking state
ChangeType::Oracle { new_oracle_config } => {
    performance_package.oracle_config = *new_oracle_config;
}

// start_unlock.rs - stores oracle-specific baseline into state
performance_package.state = PerformancePackageState::Unlocking {
    start_aggregator,      // from current oracle_config
    start_timestamp: last_updated_timestamp,
};

// complete_unlock.rs - TWAP mixes old baseline with current oracle (possibly new)
let (start_aggregator, start_timestamp) = match &performance_package.state { ... };
let current_aggregator = /* read from performance_package.oracle_config */;
let twap_price = (current_aggregator - start_aggregator) / time_passed;
```

Mitigation Suggestion

We recommend disallowing oracle changes once unlock has started. In both `propose_change` and `execute_change`, require that the performance package is not in `Unlocking` state when the change involves the oracle:

```
// In propose_change validate (and in execute_change before applying Oracle change):
if matches!(
    self.performance_package.state,
    PerformancePackageState::Unlocking { .. }
) {
    return Err(PriceBasedPerformancePackageError::InvalidPerformancePackageState.into());
}
```

For `execute_change`, enforce this specifically when `change_type` is `ChangeType::Oracle { .. }`, so that oracle config cannot be changed after `start_unlock` has run. Optionally, document that oracle changes are only valid while the package is `Locked`, and consider adding a distinct error for “cannot change oracle while unlocking” for clearer UX.

Remediation

Fixed in https://github.com/metaDAOproject/programs/commit/30307620cdb5fb3ef23a5a0a0bcf1043c2b9c460

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M9 Launch Can Be Front-Run When Base Mint Is Created Separately

MEDIUM ACKNOWLEDGED

Description

The `initialize_launch` instruction expects the base mint to already exist; it is created by the caller in a separate transaction before calling the launchpad. The launch PDA is derived only from the base mint (`seeds = [b"launch", base_mint.key().as_ref()]`) with no nonce or other distinguisher. An attacker can monitor for new mints and front-run the victim's `initialize_launch` by submitting their own `initialize_launch` for the same mint with skewed parameters or with the attacker as `launch_authority`. Once that launch account is created, the intended creator can no longer initialize a launch for that mint, and the victim loses the ability to run their launch.

```
#[account(
    init,
    payer = payer,
    space = 8 + Launch::INIT_SPACE,
    seeds = [b"launch", base_mint.key().as_ref()],
    bump
)]
pub launch: Account<'info, Launch>,

#[account(
    mut,
    mint::decimals = 6,
    mint::authority = launch_signer,
)]
pub base_mint: Account<'info, Mint>,
```

The attack works as follows:

1. The victim creates a base mint (e.g. via SPL Token). 2. The victim prepares an `initialize_launch` transaction for that mint. 3. An attacker observes the new mint (or the victim's pending transaction) and submits their own `initialize_launch` for the same `base_mint` with favourable parameters (e.g. attacker as `launch_authority`, or skewed economic parameters). 4. If the attacker's transaction is processed first, the launch PDA is initialized for that mint. 5. The victim's transaction then fails (launch account already exists), or the victim never gets to submit. Because the launch seed is fixed per mint, the victim cannot create a second launch for the same mint and is permanently locked out.

Location

/programs/v07_launchpad/src/instructions/initialize_launch.rs#L38-L105

Relevant Code

```
// initialize_launch.rs - launch PDA is uniquely determined by base_mint (no nonce)
#[account(
    init,
    payer = payer,
    space = 8 + Launch::INIT_SPACE,
    seeds = [b"launch", base_mint.key().as_ref()],
    bump
)]
pub launch: Account<'info, Launch>,

// base_mint is an existing account created by the caller before this transaction
#[account(
    mut,
    mint::decimals = 6,
    mint::authority = launch_signer,
)]
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
pub base_mint: Account<'info, Mint>,

// launch_authority is only used as a constraint; launch stores it and it can be set by whoever submits initi
alize_launch
launch_authority: ctx.accounts.launch_authority.key(),
```

Mitigation Suggestion

We recommend making mint creation and launch initialization atomic so that the launchpad program creates the base mint and the launch in the same transaction. For example, add an instruction that CPIs into the token program to create the mint (with a program-derived authority or a PDA), then initializes the launch for that newly created mint in the same instruction. That way the mint does not exist on-chain until the creator has already initialized the launch, and front-running by reusing the same mint is no longer possible.

Remediation

Marked as won't fix.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M10 `initialize_proposal` Stale Check Can Be Bypassed with Unrelated Multisig

MEDIUM FIXED

Description

The `initialize_proposal` instruction does not validate that the provided `squads_multisig` account matches the DAO's `squads_multisig`. An attacker can provide an arbitrary `squads_multisig` account with a very high `stale_transaction_index` to bypass the staleness check, allowing spam proposal creation even when the actual Squads proposal is stale for the DAO's multisig.

```
pub squads_proposal: Box<Account<'info, squads_multisig_program::Proposal>>,
pub squads_multisig: Box<Account<'info, squads_multisig_program::Multisig>>, // No constraint tying it to dao
.squads_multisig
```

The instruction checks that `squads_proposal.multisig == dao.squads_multisig` (line 59), ensuring the proposal belongs to the DAO's multisig. However, the staleness check uses the attacker-provided `squads_multisig` account instead of the DAO's actual multisig:

```
require_gt!(
    self.squads_proposal.transaction_index,
    self.squads_multisig.stale_transaction_index // Uses attacker-provided account, not dao.squads_multisig
);
```

An attacker can call `initialize_proposal` with: 1. A valid `squads_proposal` that belongs to `dao.squads_multisig` (passes the multisig check) 2. An arbitrary `squads_multisig` account (from any DAO or even a new one) with `stale_transaction_index = 78435789345789789345` 3. The staleness check passes because it compares against the attacker-provided multisig's stale index, not the DAO's actual multisig

This allows creating proposals that cannot be launched later (since `launch_proposal` correctly uses the DAO's multisig PDA), resulting in spam proposals that increment `dao.proposal_count` and create persistent junk state.

Location

/programs/futarchy/src/instructions/initialize_proposal.rs#L14-L15
/programs/futarchy/src/instructions/initialize_proposal.rs#L70-L73

Relevant Code

```
// initialize_proposal.rs - squads_multisig has no constraint tying it to dao.squads_multisig
pub squads_proposal: Box<Account<'info, squads_multisig_program::Proposal>>,
pub squads_multisig: Box<Account<'info, squads_multisig_program::Multisig>>,

// initialize_proposal.rs - stale check uses attacker-provided squads_multisig
require_gt!(
    self.squads_proposal.transaction_index,
    self.squads_multisig.stale_transaction_index // BUG: should use dao.squads_multisig
);

// initialize_proposal.rs - validates proposal belongs to DAO multisig, but not the multisig account itself
require_keys_eq!(self.squads_proposal.multisig, self.dao.squads_multisig);
```

Mitigation Suggestion

We recommend you to add a constraint to ensure the provided `squads_multisig` account matches the DAO's multisig:

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
#[account(address = dao.squads_multisig)]
pub squads_multisig: Box<Account<'info, squads_multisig_program::Multisig>>,
```

Remediation

Fixed in https://github.com/metaDAOproject/programs/commit/d35656183aaf1d7b14c4545f653c8da2e104163b

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M11 LP Position Hijack Can Lock Liquidity Until Recovery

MEDIUM FIXED

Description

The `provide_liquidity` instruction allows an attacker to overwrite the `position_authority` of an existing `AmmPosition`, temporarily locking a victim's liquidity until they recover it. The PDA for an LP position is derived from the DAO and a user-controlled `position_authority` field, but the stored `position_authority` is set to the `liquidity_provider` signer instead of the PDA seed input.

```
#[account(
    init_if_needed,
    payer = payer,
    seeds = [b"amm_position", dao.key().as_ref(), params.position_authority.key().as_ref()],
    bump,
    space = 8 + AmmPosition::INIT_SPACE,
)]
pub amm_position: Account<'info, AmmPosition>,
```

```
amm_position.set_inner(AmmPosition {
    dao: dao.key(),
    position_authority: liquidity_provider.key(), // BUG: uses liquidity_provider, not params.position_author
ity
    liquidity: amm_position.liquidity + liquidity_to_mint,
});
```

The attack works as follows:

1. A victim provides liquidity normally, creating an `AmmPosition` at PDA `[\"amm_position\", dao, victim]` with `position_authority = victim`. 2. An attacker calls `provide_liquidity` with:

- `params.position_authority = victim` (derives the victim's `AmmPosition` PDA)
- `liquidity_provider = attacker` (attacker's signer)
- Minimal `quote_amount` and `max_base_amount`

3. Because `amm_position` uses `init_if_needed`, the existing victim position is loaded. 4. `set_inner` overwrites `position_authority` with `liquidity_provider.key()` (attacker), while the PDA still uses `victim` in its seeds. 5. Now:

- The victim cannot withdraw: `withdraw_liquidity` uses `has_one = position_authority` and `seeds = [b"amm_position", dao, position_authority]`, which now expects the attacker as authority.
- The attacker cannot withdraw either: the PDA seeds require `position_authority.key() = victim` for the address, but the stored `position_authority` is attacker, so the account cannot be derived with attacker's key.

This creates a mismatch between the PDA address and the stored authority, causing a DoS on the LP position. While the victim can recover by calling `provide_liquidity` again with themselves as both `position_authority` and `liquidity_provider` (which will overwrite `position_authority` back to the victim), this requires the victim to:

- Detect that they've been attacked
- Provide additional liquidity (even minimal) to recover access
- Potentially face repeated attacks that require repeated recovery actions

Location

/programs/futarchy/src/instructions/provide_liquidity.rs#L51-L57
/programs/futarchy/src/instructions/provide_liquidity.rs#L131-L135
/programs/futarchy/src/instructions/withdraw_liquidity.rs#L45-L52

Relevant Code

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
// provide_liquidity.rs - position PDA derived from params.position_authority
#[account(
    init_if_needed,
    payer = payer,
    seeds = [b"amm_position", dao.key().as_ref(), params.position_authority.key().as_ref()],
    bump,
    space = 8 + AmmPosition::INIT_SPACE,
)]
pub amm_position: Account<'info, AmmPosition>,

// provide_liquidity.rs - stored authority uses liquidity_provider instead of params.position_authority
amm_position.set_inner(AmmPosition {
    dao: dao.key(),
    position_authority: liquidity_provider.key(), // BUG
    liquidity: amm_position.liquidity + liquidity_to_mint,
});

// withdraw_liquidity.rs - PDA and has_one both depend on position_authority
#[account(
    mut,
    seeds = [b"amm_position", dao.key().as_ref(), position_authority.key().as_ref()],
    bump,
    has_one = dao,
    has_one = position_authority,
)]
pub amm_position: Account<'info, AmmPosition>,
```

Mitigation Suggestion

We recommend you to only call `set_inner` when initializing a new position. For existing positions, only update the liquidity field:

```
if amm_position.position_authority == Pubkey::default() {
    // New account - initialize with set_inner
    amm_position.set_inner(AmmPosition {
        dao: dao.key(),
        position_authority: params.position_authority,
        liquidity: liquidity_to_mint,
    });
} else {
    // Existing account - only update liquidity
    amm_position.liquidity += liquidity_to_mint;
}
```

This ensures that once a position is created, its `position_authority` cannot be overwritten by subsequent `provide_liquidity` calls, preventing attackers from hijacking existing LP positions.

Remediation

Fixed in https://github.com/metaDAOproject/programs/commit/8748a384f9ce92ef3fafcbb9f0aac0a64863c5e8

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M12 Proposal staking minimum can be bypassed

MEDIUM ACKNOWLEDGED

Description

The `unstake_from_proposal` instruction does not validate that the proposal is still in Draft state, allowing attackers to stake tokens, launch a proposal, and immediately unstake in the same transaction or block. This bypasses the "skin-in-the-game" mechanism designed to prevent spam proposals and ensure proposers have economic commitment.

```
pub fn validate(&self, params: &UnstakeFromProposalParams) -> Result<()> {
    require_keys_eq!(self.proposal.dao, self.dao.key());

    require_gt!(params.amount, 0, FutarchyError::InvalidAmount);

    // Check if staker has enough staked
    require_gte!(
        self.stake_account.amount,
        params.amount,
        FutarchyError::InsufficientTokenBalance
    );

    Ok(())
}
```

The attack works as follows:

1. Attacker either has enough funds or takes out a flashloan 2. An attacker calls `stake_to_proposal` with the required amount of tokens (`base_to_stake`) for a Draft proposal. 3. The attacker immediately calls `launch_proposal`, which changes the proposal state from `Draft` to `Pending`. 4. The attacker immediately calls `unstake_from_proposal`. Since `unstake_from_proposal` only checks if the user has a stake balance and does not validate the proposal state, the attacker receives their tokens back instantly. 5. (optional) Attacker repays the flashloan

While `unstake_from_proposal` updates the proposal state when it's in Draft (lines 90-94), it does not prevent unstaking when the proposal is in Pending or other states. This allows attackers to launch proposals without any long-term commitment of funds, enabling spam proposals.

Location

/programs/futarchy/src/instructions/unstake_from_proposal.rs#L40-L52
/programs/futarchy/src/instructions/launch_proposal.rs#L150-L152
/programs/futarchy/src/instructions/stake_to_proposal.rs#L45-L49

Relevant Code

```
// unstake_from_proposal.rs - missing proposal state check
pub fn validate(&self, params: &UnstakeFromProposalParams) -> Result<()> {
    require_keys_eq!(self.proposal.dao, self.dao.key());

    require_gt!(params.amount, 0, FutarchyError::InvalidAmount);

    // Check if staker has enough staked
    require_gte!(
        self.stake_account.amount,
        params.amount,
        FutarchyError::InsufficientTokenBalance
    );

    Ok(())
}
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
// launch_proposal.rs - changes state from Draft to Pending
proposal.state = ProposalState::Pending;
proposal.timestamp_enqueued = clock.unix_timestamp;

// stake_to_proposal.rs - requires Draft state
require!(
    matches!(self.proposal.state, ProposalState::Draft { .. }),
    FutarchyError::ProposalNotInDraftState
);
```

Mitigation Suggestion

We recommend that you add a constraint in `unstake_from_proposal` to only allow unstaking if the proposal is still in Draft state or after it has been Finalized:

```
require!(
    matches!(self.proposal.state, ProposalState::Draft { .. })
        || matches!(self.proposal.state, ProposalState::Passed)
        || matches!(self.proposal.state, ProposalState::Failed),
    FutarchyError::ProposalNotInDraftState
);
```

This ensures that once a proposal is launched (moved to the Pending state), stakers cannot unstake until it is finalized.

Remediation

Acknowledged

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M13 `sell_tokens` Can Be Frontrun to Force Worse Price

MEDIUM FIXED

Description

The `sell_tokens` instruction calculates the sale price based on `initial_amm_quote_reserves`, which is set during bid wall initialization.

```
let total_nav: u128 = (ctx.accounts.bid_wall.initial_amm_quote_reserves
    + ctx.accounts.dao_treasury_quote_token_account.amount
    + ctx.accounts.bid_wall.quote_amount) as u128;

let amount_out_before_fee =
    (amount_in as u128 * total_nav as u128 / remaining_base as u128) as u64;
```

However, bid walls can be closed and reinitialized with a different `initial_amm_quote_reserves` value, allowing an attacker to frontrun a sell transaction and force the seller to receive a worse price.

```
// Initialize bid wall account
ctx.accounts.bid_wall.set_inner(BidWall {
    nonce: args.nonce,
    created_timestamp: clock.unix_timestamp,
    initial_amm_quote_reserves: args.initial_amm_quote_reserves,
    quote_amount: args.amount,
    fees_collected: 0,
    base_bought_amount: 0,
    seq_num: 0,
    creator: ctx.accounts.creator.key(),
    authority: ctx.accounts.authority.key(),
    dao_treasury: ctx.accounts.dao_treasury.key(),
    base_mint: ctx.accounts.base_mint.key(),
    fee_recipient: ctx.accounts.fee_recipient.key(),
    duration_seconds: args.duration_seconds,
    pda_bump: ctx.bumps.bid_wall,
});
```

The attack works as follows:

1. A user calls `sell_tokens` to sell tokens to a bid wall. 2. The user can precalculate the expected price based on the current `initial_amm_quote_reserves`. 3. An attacker viewing the transaction frontruns it by closing his bid wall and immediately reinitializing it with a lower `initial_amm_quote_reserves` value. 4. When the original `sell_tokens` transaction executes, it uses the new lower `initial_amm_quote_reserves` in the price calculation, causing the seller to receive significantly less quote tokens than expected.

While frontrunning is harder on Solana due to the widespread usage of Jito bundles, it is still possible on private nodes or when transactions are not submitted through Jito. This vulnerability allows attackers to extract value from sellers by manipulating the price calculation parameters.

Location

/programs/bid_wall/src/instructions/sell_tokens.rs#L94-L104
/programs/bid_wall/src/instructions/close_bid_wall.rs#L53-L70
/programs/bid_wall/src/instructions/initialize_bid_wall.rs#L17-L18

Relevant Code

```
// sell_tokens.rs - price calculation uses initial_amm_quote_reserves
let total_nav: u128 = (ctx.accounts.bid_wall.initial_amm_quote_reserves
    + ctx.accounts.dao_treasury_quote_token_account.amount
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
    + ctx.accounts.bid_wall.quote_amount) as u128;

let amount_out_before_fee =
    (amount_in as u128 * total_nav as u128 / remaining_base as u128) as u64;

// close_bid_wall.rs - allows closing expired/depleted bid walls
if self.bid_wall.quote_amount > 0 {
    require_gt!(
        clock.unix_timestamp,
        self.bid_wall
            .created_timestamp
            .checked_add(self.bid_wall.duration_seconds as i64)
            .unwrap(),
        BidWallError::BidWallNotExpired
    );
}

// initialize_bid_wall.rs - allows setting new initial_amm_quote_reserves
pub struct InitializeBidWallArgs {
    pub amount: u64,
    pub nonce: u64,
    pub initial_amm_quote_reserves: u64,
    pub duration_seconds: u32,
}
```

Mitigation Suggestion

We recommend you to prevent reinitialization of bid walls with the same nonce. This can be done by maintaining a global nonce tracker for each creator, which will make reinitalization at the same address impossible.

Remediation

Fixed in https://github.com/metaDAOproject/programs/commit/e9ce2e2cefa39d1b016e5e1c5cede3f2015f4cde

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M14 Fees sent to wrong recipient in case of `collect_fees` instruction

MEDIUM FIXED

Description

The `collect_fees` instruction sends fees to a hardcoded `metadao_multisig_vault::ID`, ignoring the `bid_wall.fee_recipient` field entirely. Meanwhile, `close_bid_wall` and `cancel_bid_wall` send fees to `bid_wall.fee_recipient`.

```
#[account(mut, associated_token::mint = quote_mint, associated_token::authority = metadao_multisig_vault::ID)
]
pub fee_recipient_quote_token_account: Account<'info, TokenAccount>,
```

This creates inconsistent behavior where fees collected via `collect_fees` go to MetaDAO's multisig vault, while fees collected during `close_bid_wall` or `cancel_bid_wall` go to the custom `bid_wall.fee_recipient`. If a bid wall creator sets a custom `fee_recipient` and `collect_fees` is called before `close_bid_wall`, the fees go to MetaDAO multisig instead of the intended recipient.

Location

/programs/bid_wall/src/instructions/collect_fees.rs#L36-L37
/programs/bid_wall/src/instructions/close_bid_wall.rs#L29-L30
/programs/bid_wall/src/instructions/cancel_bid_wall.rs#L28-L29

Relevant Code

```
// collect_fees.rs - hardcoded recipient
#[account(mut, associated_token::mint = quote_mint, associated_token::authority = metadao_multisig_vault::ID)
]
pub fee_recipient_quote_token_account: Account<'info, TokenAccount>,

// close_bid_wall.rs - uses stored fee_recipient
/// CHECK: used for constraints
#[account(address = bid_wall.fee_recipient)]
pub fee_recipient: UncheckedAccount<'info>,

#[account(mut, associated_token::mint = quote_mint, associated_token::authority = fee_recipient)]
pub fee_recipient_quote_token_account: Account<'info, TokenAccount>,
```

Mitigation Suggestion

We recommend you to validate that `collect_fees` sends to `bid_wall.fee_recipient` to maintain consistency with the other instructions:

```
#[account(address = bid_wall.fee_recipient)]
pub fee_recipient: UncheckedAccount<'info>,

#[account(mut, associated_token::mint = quote_mint, associated_token::authority = fee_recipient)]
pub fee_recipient_quote_token_account: Account<'info, TokenAccount>,
```

This ensures all fee collection instructions behave consistently and respect the `bid_wall.fee_recipient` field set during bid wall initialization.

Remediation

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

Fixed in https://github.com/metaDAOproject/programs/commit/9d2ff64731a7914b1ce5d5b66a943bafd40670f9

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L1 Pricebased Performance Package Propose Change Lacks Correct Event Handling

LOW FIXED

Description

We found that the propose change instruction in the price based performance package program is the only instruction that emits an event without incrementing a seq_num, and without CommonFields in the event at all.

Location

https://github.com/metaDAOproject/programs/blob/d05c432f9b67221267b0b3fd009cfd0227b4605b/programs/price_based_performance_package/src/instructions/propose_change.rs#L85-L91

Relevant Code

```
/// propose_change.rs L85-L91
        // Emit event
        emit!(ChangeProposed {
            locker: performance_package.key(),
            change_request: change_request.key(),
            proposer: proposer.key(),
            change_type,
        });
```

Mitigation Suggestion

Add commonfields to this event and increase the sequence number as well.

Remediation

Fixed in https://github.com/metaDAOproject/programs/commit/d82d3cb5427dfccf3de1a68db498f54142fab16c.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L2 `complete_launch` Creates and Funds Treasury Quote Account Even in Refunding State

LOW | ACKNOWLEDGED

Description

We found that `complete_launch` funds and initializes `treasury_quote_account` even when the launch will end up in the **refunding** state. In this path, the treasury quote account is not needed, so creating it wastes lamports.

Location

https://github.com/metaDAOproject/programs/blob/d05c432f9b67221267b0b3fd009cfd0227b4605b/programs/v07_launchpad/src/instructions/complete_launch.rs#L167-L173

Relevant Code

```
#[account(
        init_if_needed,
        payer = payer,
        associated_token::mint = quote_mint,
        associated_token::authority = squads_multisig_vault,
    )]
    pub treasury_quote_account: Box<Account<'info, TokenAccount>>,
```

Mitigation Suggestion

Only create/fund `treasury_quote_account` when the launch is completing successfully, and skip it when transitioning to refunding.

Remediation

Acknowledged.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L3 Complete Unlock Resets to Locked When No Tranche Unlocks, Enabling Griefing

LOW UNKNOWN

Description

When `complete_unlock` is called, it may unlock zero tranches (e.g. the TWAP price has not reached any tranche threshold yet). In that case the current implementation still sets the performance package state back to `Locked` and increments `seq_num`. The recipient must then call `start_unlock` again and wait another `twap_length_seconds` before any tranche can be completed. Because the instruction is not restricted to the recipient—only `token_recipient` must equal `performance_package.recipient` (an account passed in), and the only signer is the `payer`—anyone can call `complete_unlock` with the victim's recipient account. An attacker can repeatedly call `complete_unlock` when no tranche would be released (e.g. shortly after the recipient calls `start_unlock`, or when price is just below the first threshold). Each such call resets the package to `Locked`, forcing the recipient to wait another full TWAP period before progress can be made, effectively griefing them.

Relevant logic: tranches are evaluated and `tokens_to_unlock` is computed; only when `tokens_to_unlock > 0` are tokens transferred and `already_unlocked_amount` updated. Regardless of that, the state is always set back to `Locked`:

```
// Only transfer if there are tokens to unlock
if tokens_to_unlock > 0 {
    // ... transfer and update already_unlocked_amount
}

// Reset locker state back to Locked for next unlock cycle (runs even when tokens_to_unlock == 0)
performance_package.state = PerformancePackageState::Locked;
performance_package.seq_num += 1;
```

So when no tranche unlocks, the package still leaves `Unlocking` and goes to `Locked`, and the next unlock cycle requires a new `start_unlock` and another `twap_length_seconds` wait.

Location

/programs/price_based_performance_package/src/instructions/complete_unlock.rs#L146-L179
/programs/price_based_performance_package/src/instructions/complete_unlock.rs#L9-L46

Relevant Code

```
// complete_unlock handle() – state reset happens even when no tranche unlocks
if tokens_to_unlock > 0 {
    token::transfer(...)?;
    performance_package.already_unlocked_amount += tokens_to_unlock;
}
// ...
performance_package.state = PerformancePackageState::Locked;
performance_package.seq_num += 1;

// Accounts: only payer is Signer; token_recipient must match package.recipient but need not be signer
#[account(mut)]
pub payer: Signer<'info>,
/// CHECK: validated to match locker.token_recipient
#[account(address = performance_package.recipient @ ...)]
pub token_recipient: UncheckedAccount<'info>,
```

Mitigation Suggestion

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

e recommend short-circuiting when no tranche has been unlocked in this call: if `tokens_to_unlock == 0`, do not change the package state, do not increment `seq_num`, and do not emit the completion event. Return `Ok(())` immediately after the tranche loop so the package remains in `Unlocking`. That way repeated calls that unlock nothing do not reset the state, and only a call that actually unlocks at least one tranche (or that would be the natural “cycle complete” behavior) transitions the package back to `Locked`. Optionally, you could also restrict the instruction so that only the recipient (e.g. `token_recipient` as `Signer`) can call `complete_unlock`, reducing the surface for griefing.

Remediation

Issue has been acknowledged as by design.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L4 Launch Can Be Funded and Closed in the Same Second at Period End

LOW FIXED

Description

When `clock.unix_timestamp == launch.unix_timestamp_started + launch.seconds_for_launch`, both `fund` and `close_launch` are allowed. Funding uses `require_gte!(start + duration, clock)` (fund allowed when end time ≥ clock) and closing uses `require_gte!(clock, start + duration)` (close allowed when clock ≥ end time). At the boundary second both inequalities hold, so the launch can be both funded and closed in that second, leading to ambiguous behavior (e.g. a fund and a close in the same block, or ordering-dependent state).

Fund – allows funding when end ≥ clock:

```
require_gte!(
    self.launch.unix_timestamp_started.unwrap() + self.launch.seconds_for_launch as i64,
    clock.unix_timestamp,
    LaunchpadError::LaunchExpired
);
```

Close – allows closing when clock ≥ end:

```
require_gte!(
    clock.unix_timestamp,
    self.launch
        .unix_timestamp_started
        .unwrap()
        .saturating_add(self.launch.seconds_for_launch.try_into().unwrap()),
    LaunchpadError::LaunchPeriodNotOver
);
```

At the exact second where both sides are equal, both instructions succeed.

Location

/programs/v07_launchpad/src/instructions/fund.rs#L64-L68
/programs/v07_launchpad/src/instructions/close_launch.rs#L22-L29

Relevant Code

```
// fund.rs – funding allowed when (start + duration) >= clock
require_gte!(
    self.launch.unix_timestamp_started.unwrap() + self.launch.seconds_for_launch as i64,
    clock.unix_timestamp,
    LaunchpadError::LaunchExpired
);

// close_launch.rs – closing allowed when clock >= (start + duration)
require_gte!(
    clock.unix_timestamp,
    self.launch.unix_timestamp_started.unwrap()
        .saturating_add(self.launch.seconds_for_launch.try_into().unwrap()),
    LaunchpadError::LaunchPeriodNotOver
);
```

Mitigation Suggestion

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

We recommend making one of the two checks strict so that at the boundary second only one action is valid. For example, allow funding only while the launch period has not yet ended by using a strict inequality in `fund`: require `(start + duration) > clock` (i.e. `require_gt!` with the same operands). Then at the exact second when `clock == start + duration`, funding fails and only `close_launch` succeeds.

```
// In fund.rs – allow funding only when clock is strictly before end
require_gt!(
    self.launch.unix_timestamp_started.unwrap() + self.launch.seconds_for_launch as i64,
    clock.unix_timestamp,
    LaunchpadError::LaunchExpired
);
```

Remediation

Fixed in https://github.com/metaDAOproject/programs/commit/e100b45a8d83c201c6a46770e66c7edfac857feb and https://github.com/metaDAOproject/programs/commit/ad0da3870099e8d364735316bfa817a87b7e03bc

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L5 Bid Wall Price Can Be Inflated by Donating to DAO Treasury

LOW WONTFIX

Description

The `sell_tokens` instruction includes the live DAO treasury quote-token balance in `total_nav`. Because the DAO treasury quote token account is just an associated token account for `dao_treasury`, **anyone can donate quote tokens to it**, artificially increasing `total_nav` and thus the bid wall price. An attacker can donate quote tokens to `dao_treasury_quote_token_account` to push up the NAV-based price.

```
// Accounts – DAO treasury quote ATA is unconstrained beyond being the treasury's ATA
/// CHECK: used for constraints
#[account(address = bid_wall.dao_treasury)]
pub dao_treasury: AccountInfo<'info>,

#[account(associated_token::mint = quote_mint, associated_token::authority = dao_treasury)]
pub dao_treasury_quote_token_account: Account<'info, TokenAccount>;

// NAV includes full DAO treasury balance, including arbitrary donations
let total_nav: u128 = (ctx.accounts.bid_wall.initial_amm_quote_reserves
    + ctx.accounts.dao_treasury_quote_token_account.amount
    + ctx.accounts.bid_wall.quote_amount) as u128;
```

Since deposits into `dao_treasury_quote_token_account` are not controlled by the bid wall program, this allows external actors to influence the price at which the wall buys base tokens, breaking the assumption that NAV per token only reflects protocol-controlled flows and burns.

Location

/programs/bid_wall/src/instructions/sell_tokens.rs#L34-L39 /programs/bid_wall/src/instructions/sell_tokens.rs#L90-L96

Relevant Code

```
// DAO treasury quote ATA can receive arbitrary external deposits
#[account(address = bid_wall.dao_treasury)]
pub dao_treasury: AccountInfo<'info>,

#[account(associated_token::mint = quote_mint, associated_token::authority = dao_treasury)]
pub dao_treasury_quote_token_account: Account<'info, TokenAccount>;

// total_nav used to set the price per token
let total_nav: u128 = (ctx.accounts.bid_wall.initial_amm_quote_reserves
    + ctx.accounts.dao_treasury_quote_token_account.amount
    + ctx.accounts.bid_wall.quote_amount) as u128;
```

Mitigation Suggestion

We recommend removing direct dependence on the live DAO treasury ATA balance for bid wall pricing, and instead basing `total_nav` on the original amount.

Remediation

Acknowledged

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L6 Unnecessary Enforcement of ATA

LOW | ACKNOWLEDGED

Description

We found that in some instructions, the protocol forces users to use their Associated Token Accounts (ATA).

This restriction is unnecessary and reduces flexibility for users and potential integrations. For example, users may want to use a custom token account they control, rather than being forced to interact only through their ATA.

By enforcing ATA usage, the protocol limits user choice without providing additional safety, since ownership and mint checks can be performed directly on any token account.

Location

https://github.com/metaDAOproject/programs/blob/d05c432f9b67221267b0b3fd009cfd0227b4605b/programs/bid_wall/src/instructions/initialize_bid_wall.rs#L49-L50

https://github.com/metaDAOproject/programs/blob/d05c432f9b67221267b0b3fd009cfd0227b4605b/programs/bid_wall/src/instructions/sell_tokens.rs#L25-L29

Relevant Code

```
#[account(mut, associated_token::mint = quote_mint, associated_token::authority = creator)]
    pub creator_quote_token_account: Account<'info, TokenAccount>,
```

```
    #[account(mut, associated_token::mint = base_mint, associated_token::authority = user)]
    pub user_token_account: Account<'info, TokenAccount>,


    #[account(mut, associated_token::mint = quote_mint, associated_token::authority = user)]
    pub user_quote_token_account: Account<'info, TokenAccount>,
```

Mitigation Suggestion

Allow users to provide any valid token account they control, rather than forcing the use of ATA accounts.

Remediation

Acknowledged.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L7 Performance Package Token Vault Not Closed After Burn

LOW | ACKNOWLEDGED

Description

We found that `performance_package_token_vault` is not closed after `burn_performance_package.rs` is executed. As a result, the rent locked in the vault account is not reclaimed, leaving unused accounts and stranded lamports.

Location

https://github.com/metaDAOproject/programs/blob/d05c432f9b67221267b0b3fd009cfd0227b4605b/programs/price_based_performance_package/src/instructions/burn_performance_package.rs#L14C13-L15C1

Relevant Code

Mitigation Suggestion

Close `performance_package_token_vault` during burn.

Remediation

Acknowledged.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L8 Unused `unlocked` State Causes Incorrect Performance Package Finalization

LOW FIXED

Description

We found that `PerformancePackageState::unlocked` is currently unused. In the complete unlock flow, the state moves directly to **locking** without checking whether all tranches are actually unlocked. As a result, a performance package can transition to the locking state even when all tranches have been unlocked, leading to incorrect package state and broken lifecycle logic.

Location

https://github.com/metaDAOproject/programs/blob/d05c432f9b67221267b0b3fd009cfd0227b4605b/programs/price_based_performance_package/src/instructions/complete_unlock.rs#L178-L179

Relevant Code

Mitigation Suggestion

Use `PerformancePackageState::unlocked` properly and transition to unlocking after verifying all tranches are unlocked.

Remediation

Fixed in https://github.com/metaDAOproject/programs/commit/813a864823b531e757107c833707add0620b09cc by removing the unlocked state.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L9 Proposal Can Be Traded After Expiry Before Finalization

LOW UNKNOWN

Description

We found that proposals can be traded in the conditional Swap and spot Swap instructions even when the proposal trading time has passed as long as the proposal hasn't been finalized yet. It would be correct to prohibit trading the proposal once `duration_in_seconds` has passed even when it is still in `Pending` state.

Location

https://github.com/metaDAOproject/programs/blob/d05c432f9b67221267b0b3fd009cfd0227b4605b/programs/futarchy/src/instructions/conditional_swap.rs#L74-L89

Relevant Code

```
/// conditional_swap.rs L74-L89
    pub fn validate(&self, params: &ConditionalSwapParams) -> Result<()> {
        require_neq!(params.market, Market::Spot);

        require_gte!(
            self.user_input_account.amount,
            params.input_amount,
            FutarchyError::InsufficientBalance
        );

        require_eq!(
            self.proposal.state,
            ProposalState::Pending,
            FutarchyError::ProposalNotActive
        );
        Ok(())
    }
```

Mitigation Suggestion

Add a check in both Swap instructions that check the current time, and block trade execution if the proposal trading time has ended but the proposal hasn't been finalized yet.

Remediation

Acknowledged.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L10 DAO parameters can be modified during active futarchy markets, undermining trader assumptions

LOW FIXED

Description

The `update_dao` instruction allows modification of critical DAO parameters at any time, including when active futarchy markets are running (when `dao.amm.state` is in `PoolState::Futarchy` state). This creates a serious issue where traders who have taken positions in pass/fail markets based on specific market parameters can have those parameters changed mid-flight, fundamentally altering the assumptions they used to make their trades.

When a DAO is in futarchy state (has an active proposal with pass/fail markets), traders are actively trading based on parameters such as:

1. `pass_threshold_bps` - The threshold at which a proposal passes. If this is lowered from 500 bps to 100 bps after traders have positioned themselves, those who bought fail tokens expecting the higher threshold would be disadvantaged.

2. `twap_initial_observation` **and** `twap_max_observation_change_per_update` - These are critical manipulation-resistance parameters. As the code comments note, an attacker could exploit incorrect initial observations. Changing these mid-market could either enable or prevent manipulation strategies that traders were counting on being fixed.

3. `twap_start_delay_seconds` - This affects when TWAP calculation begins, which directly impacts proposal outcomes. Changing this during an active market could shift outcomes.

4. `seconds_per_proposal` - Changing the duration of a proposal mid-flight would alter trader time horizons and strategies.

5. `team_sponsored_pass_threshold_bps` - Similar to pass_threshold_bps, this affects proposal outcomes.

The UpdateDao instruction (lines 26-97) performs no checks on whether the DAO is currently in a futarchy state before allowing parameter updates. The instruction only validates that `dao.squads_multisig_vault` is a signer, but does not check `dao.amm.state`.

Security Impact:

- Traders lose the ability to make informed decisions as the rules can change during active trading
- Market participants can be rug-pulled by governance changing parameters before proposal finalization
- Price discovery is undermined when fundamental market parameters are not fixed
- This could be exploited by malicious actors who see a proposal trending against their interests and modify parameters to influence the outcome

Location

https://github.com/metaDAOproject/programs/blob/d05c432f9b67221267b0b3fd009cfd0227b4605b/programs/futarchy/src/instructions/update_dao.rs#L30-L32

Relevant Code

```
        dao.set_inner(Dao {
            amm: dao.amm.clone(),
            nonce: dao.nonce,
            dao_creator: dao.dao_creator,
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
            pda_bump: dao.pda_bump,
            squads_multisig: dao.squads_multisig,
            squads_multisig_vault: dao.squads_multisig_vault,
            base_mint: dao.base_mint,
            quote_mint: dao.quote_mint,
            proposal_count: dao.proposal_count,
            pass_threshold_bps: dao_params
                .pass_threshold_bps
                .unwrap_or(dao.pass_threshold_bps),
            seconds_per_proposal: dao_params
                .seconds_per_proposal
                .unwrap_or(dao.seconds_per_proposal),
            twap_initial_observation: dao_params
                .twap_initial_observation
                .unwrap_or(dao.twap_initial_observation),
            twap_max_observation_change_per_update: dao_params
                .twap_max_observation_change_per_update
                .unwrap_or(dao.twap_max_observation_change_per_update),
            twap_start_delay_seconds: dao_params
                .twap_start_delay_seconds
                .unwrap_or(dao.twap_start_delay_seconds),
```

Mitigation Suggestion

Add a check at the beginning of the `UpdateDao::handle` function to ensure the DAO is not currently in a futarchy state:

```
impl UpdateDao<'_> {
    pub fn handle(ctx: Context<Self>, dao_params: UpdateDaoParams) -> Result<()> {
        let dao = &mut ctx.accounts.dao;

        // Prevent parameter updates during active futarchy markets
        match dao.amm.state {
            PoolState::Futarchy { .. } => {
                return Err(FutarchyError::CannotUpdateDaoDuringFutarchy.into());
            }
            PoolState::Spot { .. } => {
                // Safe to update when only spot market exists
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
            }
        }

        // ... rest of implementation
    }
}
```

This ensures that DAO parameter changes can only be made when no active futarchy markets exist, protecting traders who have positioned themselves based on the current parameters. Parameters can still be updated between proposals when the DAO is in spot-only mode.

Remediation

Fixed in https://github.com/metaDAOproject/programs/commit/92875b1114309f65cefcec59e258c046e0a9e911 and https://github.com/metaDAOproject/programs/commit/66e4a0226c3d42437200e5a2365337d6bc7c1999.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L11 Premine Cap Check Ignores `additional_tokens_amount`

LOW FIXED

Description

The premine cap is enforced only on `performance_package_token_amount`; `additional_tokens_amount` is not included. Both amounts are part of the premine (minted at launch and allocated to the team / additional recipient), so the effective premine can exceed `MAX_PREMINE`.

```
require_gte!(
    MAX_PREMINE,
    args.performance_package_token_amount,
    LaunchpadError::InvalidPriceBasedPremineAmount
);
```

A launch can set e.g. `performance_package_token_amount = MAX_PREMINE` and `additional_tokens_amount` to any non-zero value (with a recipient). Total premine then exceeds `MAX_PREMINE`, bypassing the intended cap.

Location

/programs/v07_launchpad/src/instructions/initialize_launch.rs#L158-L162
/programs/v07_launchpad/src/instructions/initialize_launch.rs#L314-L319

Relevant Code

```
// initialize_launch.rs - only performance_package_token_amount is checked
require_gte!(
    MAX_PREMINE,
    args.performance_package_token_amount,
    LaunchpadError::InvalidPriceBasedPremineAmount
);
```

Mitigation Suggestion

We recommend enforcing the premine cap on the sum of both components:

```
let total_premine = args
    .performance_package_token_amount
    .saturating_add(args.additional_tokens_amount);

require_gte!(
    MAX_PREMINE,
    total_premine,
    LaunchpadError::InvalidPriceBasedPremineAmount
);
```

This ensures the total premine (performance package + additional tokens) cannot exceed `MAX_PREMINE`.

Remediation

Fixed in https://github.com/metaDAOproject/programs/commit/efaca01f5b08838aeea8738f59269f9cf8213cfa

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L12 MAX_PREMINE Allows More Than 50% of Total Supply

LOW FIXED

Description

The constant `MAX_PREMINE` is documented as "Max 50% premine" but its value (15M tokens) is larger than the fixed non■premine allocation (12.9M tokens). As a result, a launch can have a premine share of more than 50% of total supply, contradicting the documented cap.

```
/// 10M tokens with 6 decimals
pub const TOKENS_TO_PARTICIPANTS: u64 = 10_000_000 * TOKEN_SCALE;
/// 20% to liquidity
pub const TOKENS_TO_FUTARCHY_LIQUIDITY: u64 = 2_000_000 * TOKEN_SCALE;
/// 3M tokens to single-sided DammV2 liquidity
pub const TOKENS_TO_DAMM_V2_LIQUIDITY: u64 = TOKENS_TO_DAMM_V2_LIQUIDITY_UNSCALED * TOKEN_SCALE;
/// we need this to prevent overflow
pub const TOKENS_TO_DAMM_V2_LIQUIDITY_UNSCALED: u64 = 900_000;

/// Max 50% premine
pub const MAX_PREMINE: u64 = 15_000_000 * TOKEN_SCALE;
```

Total supply is `performance_package_token_amount + additional_tokens_amount + TOKENS_TO_PARTICIPANTS + TOKENS_TO_FUTARCHY_LIQUIDITY + TOKENS_TO_DAMM_V2_LIQUIDITY`. The fixed part is 10M + 2M + 0.9M = 12.9M tokens. With `MAX_PREMINE = 15M`, premine (performance package + additional tokens) can be up to 15M, so total supply can be 15M + 12.9M = 27.9M, and premine can be 15/27.9 ≈ 53.8%, which exceeds 50%.

Location

/programs/v07_launchpad/src/lib.rs#L32-L42
/programs/v07_launchpad/src/instructions/initialize_launch.rs#L314-L319

Relevant Code

```
// lib.rs - comment says 50% but 15M > 12.9M fixed allocation
/// Max 50% premine
pub const MAX_PREMINE: u64 = 15_000_000 * TOKEN_SCALE;

// Fixed allocation = 10M + 2M + 0.9M = 12.9M
TOKENS_TO_PARTICIPANTS + TOKENS_TO_FUTARCHY_LIQUIDITY + TOKENS_TO_DAMM_V2_LIQUIDITY

// initialize_launch.rs - total supply = premine + fixed allocation
args.performance_package_token_amount
    + args.additional_tokens_amount
    + TOKENS_TO_PARTICIPANTS
    + TOKENS_TO_FUTARCHY_LIQUIDITY
    + TOKENS_TO_DAMM_V2_LIQUIDITY,
```

Mitigation Suggestion

To enforce a true maximum 50% premine, cap premine at the size of the fixed allocation. For example, set `MAX_PREMINE` so that premine cannot exceed the fixed 12.9M tokens (e.g. `MAX_PREMINE = 12_900_000 * TOKEN_SCALE`), or derive the cap from the fixed constants so that `premine / (premine + fixed_allocation) <= 0.5` and update the comment to match the chosen constant.

Remediation

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

Fixed in https://github.com/metaDAOproject/programs/commit/efaca01f5b08838aeea8738f59269f9cf8213cfa

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L13 Performance Package Over-allocates Rent for Unused Tranches

LOW | ACKNOWLEDGED

Description

The `PerformancePackage` account is allocated space for up to 10 tranches (`MAX_TRANCHES = 10`), but the initialization in `initialize_performance_package` always creates exactly 5 tranches. This results in unnecessary rent being paid for 5 unused tranche slots.

```
#[account]
#[derive(InitSpace, Debug)]
pub struct PerformancePackage {
    /// The tranches that make up the performance package
    #[max_len(MAX_TRANCHES)]
    pub tranches: Vec<StoredTranche>,
    // ... other fields
}
```

Each `StoredTranche` occupies 25 bytes (`price_threshold: u128` = 16 bytes, `token_amount: u64` = 8 bytes, `is_unlocked: bool` = 1 byte). With `MAX_TRANCHES = 10` but only 5 tranches being initialized, 5 unused tranche slots result in 125 bytes of unnecessary account space allocation, for which rent must be paid.

Location

/programs/price_based_performance_package/src/state/performance_package.rs#L54-L56
/programs/price_based_performance_package/src/constants.rs#L3-L4
/programs/v07_launchpad/src/instructions/initialize_performance_package.rs#L152-L174

Relevant Code

```
// performance_package.rs - max_len allows up to 10 tranches
#[account]
#[derive(InitSpace, Debug)]
pub struct PerformancePackage {
    /// The tranches that make up the performance package
    #[max_len(MAX_TRANCHES)]
    pub tranches: Vec<StoredTranche>,
    // ...
}

// constants.rs - MAX_TRANCHES is set to 10
#[constant]
pub const MAX_TRANCHES: usize = 10;

// initialize_performance_package.rs - always creates exactly 5 tranches
InitializePerformancePackageParams {
    tranches: vec![
        Tranche {
            price_threshold: launch_price_1e12 * 2,
            token_amount: self.launch.performance_package_token_amount / 5,
        },
        Tranche {
            price_threshold: launch_price_1e12 * 4,
            token_amount: self.launch.performance_package_token_amount / 5,
        },
        Tranche {
            price_threshold: launch_price_1e12 * 8,
            token_amount: self.launch.performance_package_token_amount / 5,
        },
        Tranche {
            price_threshold: launch_price_1e12 * 16,
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
            token_amount: self.launch.performance_package_token_amount / 5,
        },
        Tranche {
            price_threshold: launch_price_1e12 * 32,
            token_amount: self.launch.performance_package_token_amount / 5,
        },
    ],
    // ...
}
```

Mitigation Suggestion

We recommend you to either reduce `MAX_TRANCHES` to 5 to match the actual usage.

```
#[constant]
pub const MAX_TRANCHES: usize = 5;
```

This will reduce the account size by 125 bytes (5 unused tranches × 25 bytes per tranche), resulting in lower rent costs for all `PerformancePackage` accounts.

Remediation

Acknowledged

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L14 AMM Position Rent Can Never Be Reclaimed

LOW | ACKNOWLEDGED

Description

The `provide_liquidity` instruction creates a per-(dao, position_authority) `AmmPosition` PDA using `init_if_needed`, funded by the user via `payer = payer`. However, there is no path anywhere in the program that closes this `AmmPosition` or returns its rent-exempt lamports to the user. The `withdraw_liquidity` instruction reduces `amm_position.liquidity` when a user withdraws, but leaves the `AmmPosition` allocated forever.

```
#[account(
    init_if_needed,
    payer = payer,
    seeds = [b"amm_position", dao.key().as_ref(), params.position_authority.key().as_ref()],
    bump,
    space = 8 + AmmPosition::INIT_SPACE,
)]
pub amm_position: Account<'info, AmmPosition>,
```

When a user provides liquidity, they pay rent for an `AmmPosition` at PDA `["amm_position", dao, position_authority]`. When they later fully withdraw their liquidity (bringing `amm_position.liquidity` to 0), the account remains allocated and owned by the program with no mechanism to reclaim or reuse it. Repeated liquidity provision across different DAOs or position authorities will accumulate many empty `AmmPosition` PDAs that are still rent-funded, effectively leaking SOL from liquidity providers and bloating on-chain state.

Location

/programs/futarchy/src/instructions/provide_liquidity.rs#L50-L57
/programs/futarchy/src/instructions/withdraw_liquidity.rs#L45-L52
/programs/futarchy/src/instructions/withdraw_liquidity.rs#L116-L117

Relevant Code

```
// provide_liquidity.rs - AmmPosition is created and rent-paid by the user
#[account(
    init_if_needed,
    payer = payer,
    seeds = [b"amm_position", dao.key().as_ref(), params.position_authority.key().as_ref()],
    bump,
    space = 8 + AmmPosition::INIT_SPACE,
)]
pub amm_position: Account<'info, AmmPosition>,

// amm_position.rs - simple account with no close semantics
#[account]
#[derive(InitSpace)]
pub struct AmmPosition {
    pub dao: Pubkey,
    pub position_authority: Pubkey,
    pub liquidity: u128,
}

// withdraw_liquidity.rs - liquidity is reduced, but account is never closed
// Update the AMM position
amm_position.liquidity -= liquidity_to_withdraw;
```

Mitigation Suggestion

We recommend you to close AMM positions when `liquidity` reaches zero.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

Remediation

Acknowledged

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L15 `provide_liquidity` overcharges base amount by 1 unit in case of no rounding

LOW FIXED

Description

In the `provide_liquidity` instruction, the base token amount to deposit is computed using integer division and then unconditionally incremented by 1 before casting to `u64`. This introduces a systematic off-by-one error that can cause users to overpay 1 unit of the base token whenever the division does not round.

```
// base_amount calculation for existing liquidity
let base_amount: u64 = (((quote_amount as u128 * base_reserves) / quote_reserves) + 1)
    .try_into()
    .map_err(|_| FutarchyError::CastingOverflow)?;
```

If `(quote_amount * base_reserves) / quote_reserves` already divides evenly (no rounding needed), the `+ 1` forces the user to deposit one extra unit of the base token, which is unnecessary as no rounding error needs to be accounted for.

Location

/programs/futarchy/src/instructions/provide_liquidity.rs#L95-L101

Relevant Code

```
// base_amount is always rounded up by 1, even when division is exact
let quote_reserves = spot.quote_reserves as u128;
let base_reserves = spot.base_reserves as u128;

let base_amount: u64 = (((quote_amount as u128 * base_reserves) / quote_reserves) + 1)
    .try_into()
    .map_err(|_| FutarchyError::CastingOverflow)?;
```

Mitigation Suggestion

We recommend you to perform a ceiling division:

```
let numerator = quote_amount as u128 * base_reserves;
let base_amount_u128 = (numerator + quote_reserves - 1) / quote_reserves; // ceil(numerator / quote_reserves)

let base_amount: u64 = base_amount_u128
    .try_into()
    .map_err(|_| FutarchyError::CastingOverflow)?;
```

Remediation

Fixed in https://github.com/metaDAOproject/programs/commit/3e4b4e77383b29325daef07ba37758b0124c4c98

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L16 `provide_liquidity` Ignores `min_liquidity` slippage parameter for First LP

LOW FIXED

Description

The `provide_liquidity` instruction takes a user-provided `min_liquidity` parameter intended to enforce a minimum amount of LP tokens minted (slippage protection). This parameter is correctly enforced for subsequent liquidity providers, but it is completely ignored for the first liquidity provider when `total_liquidity == 0`.

```
let ProvideLiquidityParams {
    quote_amount,
    max_base_amount,
    min_liquidity,
    position_authority: _,
} = params;

let total_liquidity = dao.amm.total_liquidity;
let PoolState::Spot { ref mut spot } = dao.amm.state else {
    // TODO: check that pool is already in right state
    unreachable!();
};

let (liquidity_to_mint, base_amount) = if total_liquidity > 0 {
    // require!(min_lp_tokens > 0, AmmError::ZeroMinLpTokens);
    require_gt!(min_liquidity, 0);
    …
    let liquidity_to_mint = (quote_amount as u128 * total_liquidity) / quote_reserves;
    …
    require_gte!(
        liquidity_to_mint,
        min_liquidity,
        // AmmError::AddLiquiditySlippageExceeded
    );

    (liquidity_to_mint, base_amount)
} else {
    // equivalent to $0.1 if the quote is USDC, here for rounding
    require_gte!(quote_amount, MIN_QUOTE_LIQUIDITY);

    let base_amount = max_base_amount;

    let initial_liquidity = quote_amount as u128 * 1_000_000_000;

    (initial_liquidity, base_amount)
};
```

When `total_liquidity > 0`, the code enforces `min_liquidity` via `require_gt!(min_liquidity, 0)` and `require_gte!(liquidity_to_mint, min_liquidity, …)`. However, in the `else` branch (first LP), `min_liquidity` is never read or checked, and `initial_liquidity` is set unconditionally to `quote_amount * 1_000_000_000`. This means the first LP's slippage bound is silently ignored, and they may receive fewer LP tokens than expected based on their `min_liquidity` parameter.

Location

/programs/futarchy/src/instructions/provide_liquidity.rs#L78-L126

Relevant Code

```
// Parameter unpacking, including min_liquidity
let ProvideLiquidityParams {
    quote_amount,
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
    max_base_amount,
    min_liquidity,
    position_authority: _,
} = params;

let total_liquidity = dao.amm.total_liquidity;

let (liquidity_to_mint, base_amount) = if total_liquidity > 0 {
    require_gt!(min_liquidity, 0);
    …
    let liquidity_to_mint = (quote_amount as u128 * total_liquidity) / quote_reserves;
    …
    require_gte!(liquidity_to_mint, min_liquidity, /* slippage check */);
    (liquidity_to_mint, base_amount)
} else {
    require_gte!(quote_amount, MIN_QUOTE_LIQUIDITY);

    let base_amount = max_base_amount;
    let initial_liquidity = quote_amount as u128 * 1_000_000_000;

    (initial_liquidity, base_amount) // min_liquidity is not used here
};
```

Mitigation Suggestion

We recommend you to enforce `min_liquidity` consistently for the first liquidity provider as well. For example:

```
} else {
    require_gte!(quote_amount, MIN_QUOTE_LIQUIDITY);

    let base_amount = max_base_amount;
    let initial_liquidity = quote_amount as u128 * 1_000_000_000;

    require_gte!(
        initial_liquidity,
        min_liquidity,
        FutarchyError::InsufficientLiquidity // or a dedicated slippage error
    );

    (initial_liquidity, base_amount)
}
```

Remediation

Fixed in https://github.com/metaDAOproject/programs/commit/f01a1bab1f8c27f88b03b439d9011da2e1dcd8b2

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L17 Stake Account Rent Is Never Reclaimed

LOW | ACKNOWLEDGED

Description

The `stake_to_proposal` instruction creates a per-(proposal, staker) `StakeAccount` PDA using `init_if_needed`, funded by the user via `payer = payer`. However, there is no path anywhere in the program that ever closes this `StakeAccount` or returns its rent-exempt lamports to the user. The `unstake_from_proposal` instruction reduces `stake_account.amount` when a user unstakes, but leaves the `StakeAccount` allocated forever.

Over time, every user who stakes to proposals accumulates an increasing number of empty `StakeAccount` PDAs that remain rent-funded. This results in:

- Permanent loss of the SOL used to pay rent for these accounts
- Unbounded state growth on-chain, as these accounts are never reclaimed or reused

Location

/programs/futarchy/src/instructions/stake_to_proposal.rs#L24-L35
/programs/futarchy/src/instructions/unstake_from_proposal.rs#L28-L33
/programs/futarchy/src/instructions/unstake_from_proposal.rs#L90-L99

Relevant Code

```
// stake_to_proposal.rs – StakeAccount is created and rent-paid by the user
#[account(
    init_if_needed,
    payer = payer,
    seeds = [b\"stake\", proposal.key().as_ref(), staker.key().as_ref()],
    bump,
    space = 8 + StakeAccount::INIT_SPACE,
)]
pub stake_account: Box<Account<'info, StakeAccount>>;

// stake_account.rs – simple account with no close semantics
#[account]
#[derive(InitSpace)]
pub struct StakeAccount {
    pub proposal: Pubkey,
    pub staker: Pubkey,
    pub amount: u64,
    pub bump: u8,
}

// unstake_from_proposal.rs – stake amount is reduced, but account is never closed
// Update stake account
stake_account.amount = stake_account.amount.saturating_sub(amount);
```

Mitigation Suggestion

We recommend allowing the user to reclaim rent for empty stake accounts by closing them when the `amount` reaches zero. The easiest approach is to close the account in `unstake_from_proposal` if reducing the amount leads to a zero amount.

Remediation

Acknowledged

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L18 Zero Address `position_authority` Can Permanently Lock Funds

LOW WONTFIX

Description

The `provide_liquidity` instruction does not validate that `params.position_authority` is not the zero address (`Pubkey::default()`). If a user accidentally provides the system program address as the `position_authority`, the resulting LP position cannot be withdrawn, permanently locking the funds.

```
pub struct ProvideLiquidityParams {
    pub quote_amount: u64,
    pub max_base_amount: u64,
    pub min_liquidity: u128,
    pub position_authority: Pubkey, // No validation that this is not Pubkey::default()
}
```

The `withdraw_liquidity` instruction requires `position_authority` to be a `Signer` and uses it in the PDA seeds:

```
#[account(
    mut,
    seeds = [b"amm_position", dao.key().as_ref(), position_authority.key().as_ref()],
    bump,
    has_one = dao,
    has_one = position_authority,
)]
pub amm_position: Account<'info, AmmPosition>,
```

If `params.position_authority = Pubkey::default()` (the system program address), the position is created with a PDA derived from the system program. However, no one can sign as the system program, so the `has_one = position_authority` constraint in `withdraw_liquidity` can never be satisfied. This permanently locks all liquidity in that position with no recovery mechanism.

Location

/programs/futarchy/src/instructions/provide_liquidity.rs#L4-L14
/programs/futarchy/src/instructions/withdraw_liquidity.rs#L45-L52

Relevant Code

```
// provide_liquidity.rs - no validation of position_authority
pub struct ProvideLiquidityParams {
    pub quote_amount: u64,
    pub max_base_amount: u64,
    pub min_liquidity: u128,
    pub position_authority: Pubkey, // Can be Pubkey::default()
}

// withdraw_liquidity.rs - requires position_authority to be a signer
#[account(
    mut,
    seeds = [b"amm_position", dao.key().as_ref(), position_authority.key().as_ref()],
    bump,
    has_one = dao,
    has_one = position_authority, // Requires position_authority to sign
)]
pub amm_position: Account<'info, AmmPosition>,

pub position_authority: Signer<'info>,
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

Mitigation Suggestion

We recommend you to add a validation check at the beginning of the `handle` function to ensure `position_authority` is not the zero address:

```
pub fn handle(ctx: Context<Self>, params: ProvideLiquidityParams) -> Result<()> {
    require_neq!(
        params.position_authority,
        Pubkey::default(),
        FutarchyError::InvalidPositionAuthority
    );

    // ... rest of the function
}
```

This prevents users from accidentally or maliciously creating positions with an unrecoverable `position_authority`, ensuring all LP positions can be withdrawn.

Remediation

Acknowledged

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L19 `sell_tokens` Can Result in Zero Output Due to Rounding

LOW FIXED

Description

The `sell_tokens` instruction uses integer division for price calculations, which can round down to zero. If the calculated output amount is 0, the user will burn their tokens but receive zero quote tokens in return.

```
let amount_out_before_fee =
    (amount_in as u128 * total_nav as u128 / remaining_base as u128) as u64;

let amount_out_after_fee =
    ((10_000_u128 - FEE_BPS as u128) * amount_out_before_fee as u128 / 10_000_u128) as u64;
```

When `amount_in * total_nav / remaining_base` results in a value less than 1, `amount_out_before_fee` will be 0 due to integer division truncation. Similarly, if `amount_out_before_fee` is a small positive value, the fee calculation `((10_000 - FEE_BPS) * amount_out_before_fee / 10_000)` can also round down to 0. In either case, the user's tokens are burned but they receive no quote tokens, resulting in a complete loss of value.

This can occur when:

- A user attempts to sell a very small amount of tokens
- The bid wall has a low NAV relative to remaining base tokens
- The calculation results in a fractional value that gets truncated to 0

There is no validation to ensure `amount_out_after_fee` is greater than 0 before burning the user's tokens, allowing this loss scenario to occur.

Location

/programs/bid_wall/src/instructions/sell_tokens.rs#L103-L115

Relevant Code

```
let amount_out_before_fee =
    (amount_in as u128 * total_nav as u128 / remaining_base as u128) as u64;

require_gte!(
    ctx.accounts.bid_wall.quote_amount,
    amount_out_before_fee,
    BidWallError::InsufficientQuoteReserves
);

let amount_out_after_fee =
    ((10_000_u128 - FEE_BPS as u128) * amount_out_before_fee as u128 / 10_000_u128) as u64;

let fee = amount_out_before_fee - amount_out_after_fee;

// Burn base tokens
token::burn(
    CpiContext::new(
        ctx.accounts.token_program.to_account_info(),
        Burn {
            mint: ctx.accounts.base_mint.to_account_info(),
            from: ctx.accounts.user_token_account.to_account_info(),
            authority: ctx.accounts.user.to_account_info(),
        },
    ),
    amount_in,
)?;
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
// Transfer quote tokens to user
token::transfer(
    CpiContext::new_with_signer(
        ctx.accounts.token_program.to_account_info(),
        Transfer {
            from: ctx.accounts.bid_wall_quote_token_account.to_account_info(),
            to: ctx.accounts.user_quote_token_account.to_account_info(),
            authority: ctx.accounts.bid_wall.to_account_info(),
        },
        &[&[
            b"bid_wall",
            ctx.accounts.base_mint.key().as_ref(),
            ctx.accounts.bid_wall.creator.as_ref(),
            ctx.accounts.bid_wall.nonce.to_le_bytes().as_ref(),
            &[ctx.accounts.bid_wall.pda_bump],
        ]],
    ),
    amount_out_after_fee,
)?;
```

Mitigation Suggestion

We recommend you to add a validation check to ensure `amount_out_after_fee` is greater than 0 before proceeding with the token burn and transfer:

```
require_gt!(
    amount_out_after_fee,
    0,
    BidWallError::InsufficientOutputAmount
);
```

This check should be placed after calculating `amount_out_after_fee` and before burning the user's tokens.

Remediation

Fixed in https://github.com/metaDAOproject/programs/commit/6800e457fa7df22b1f978b7c1ddb85a8aa2740be

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L20 `close_bid_wall` is missing authority signature allowing for rent theft

LOW FIXED

Description

The `authority` account is declared as `UncheckedAccount<'info>` instead of `Signer<'info>`, meaning it does not need to sign the transaction. The `has_one = authority` constraint only verifies that the provided `authority` account's public key matches `bid_wall.authority`, but does not require the authority to sign the transaction.

```
/// CHECK: used for constraints
#[account(address = bid_wall.authority)]
pub authority: UncheckedAccount<'info>,
```

All token transfers in the instruction use the `bid_wall` PDA as the authority (signed with PDA seeds), so the `authority` account never needs to sign. An attacker can identify an expired or depleted bid wall and call `close_bid_wall` with the correct `authority` account (public key only, no signature needed), the correct associated token accounts, and any `payer` account. The instruction will execute successfully, allowing the caller to cancel someone else‘s bidwall and steal their rent (20c).

Location

https://github.com/metaDAOproject/programs/blob/d05c432f9b67221267b0b3fd009cfd0227b4605b/programs/bid_wall/src/instructions/close_bid_wall.rs#L24-L26

Relevant Code

```
#[account(
    mut,
    close=payer,
    has_one = authority
)]
pub bid_wall: Account<'info, BidWall>,

#[account(mut)]
pub payer: Signer<'info>,

/// CHECK: used for constraints
#[account(address = bid_wall.authority)]
pub authority: UncheckedAccount<'info>,
```

Mitigation Suggestion

We recommend you to change the `authority` account to require a signature:

```
// Authority must sign to prevent unauthorized bid wall closure
#[account(address = bid_wall.authority)]
pub authority: Signer<'info>,
```

This ensures only the authority can close bid walls.

Remediation

Fixed in https://github.com/metaDAOproject/programs/commit/d7fc89a7dcfb66c05106d0bcb07959c66c8e78d6

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I1 Integer Overflow in Total Token Amount Calculation During Performance Package Initialization

INFO FIXED

Description

The `initialize_performance_package` instruction calculates the total token amount by summing all tranche token amounts without using checked arithmetic (line 126). Since each tranche can contain up to `u64::MAX` tokens and the system allows up to 10 tranches (`MAX_TRANCHES = 10`), this summation can silently overflow in release builds.

When an overflow occurs, the `total_token_amount` wraps around to a smaller value. This creates a severe discrepancy between:

1. The actual tokens transferred to the vault (line 142)

2. The `total_token_amount` stored in the performance package state (line 155)

Attack Scenario:

An attacker could create a performance package with multiple tranches where the sum exceeds `u64::MAX`. For example:

- Tranche 1: `u64::MAX - 1000`
- Tranche 2: `2000`
- Sum with overflow: `999` (wraps around)

The attacker would only need to deposit 999 tokens (due to the check on line 130), but the individual tranches would collectively claim `u64::MAX + 1000` tokens. When unlocking tranches later, the attacker would try to drain tokens beyond what they deposited, attempting to steal funds from the vault. However, the vault would only have the actual deposited amount, leading just to an accounting mismatch.

Location

https://github.com/metaDAOproject/programs/blob/d05c432f9b67221267b0b3fd009cfd0227b4605b/programs/price_based_performance_package/src/instructions/initialize_performance_package.rs#L126

Relevant Code

```
let total_token_amount = tranches.iter().map(|tranche| tranche.token_amount).sum();

require_gt!(total_token_amount, 0);

require_gte!(grantor_token_account.amount, total_token_amount);

// Transfer tokens from user to locker
let transfer_ctx = CpiContext::new(
    token_program.to_account_info(),
    token::Transfer {
        from: grantor_token_account.to_account_info(),
        to: performance_package_token_vault.to_account_info(),
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
        authority: grantor.to_account_info(),
    },
);

token::transfer(transfer_ctx, total_token_amount)?;
```

Mitigation Suggestion

Replace the unchecked `.sum()` operation with explicit checked arithmetic that returns an error on overflow:

```
let total_token_amount = tranches
    .iter()
    .try_fold(0u64, |acc, tranche| {
        acc.checked_add(tranche.token_amount)
            .ok_or(PriceBasedPerformancePackageError::TotalTokenAmountOverflow)
    })?;
```

This ensures that any attempt to create a performance package with a total token amount exceeding `u64::MAX` will fail immediately with a clear error, preventing the accounting discrepancy.

Additionally, consider adding a custom error variant:

```
#[error_code]
pub enum PriceBasedPerformancePackageError {
    // ... existing errors ...
    #[msg("Total token amount calculation would overflow")]
    TotalTokenAmountOverflow,
}
```

Remediation

Fixed in https://github.com/metaDAOproject/programs/commit/b197b81992529ca48b8d45e671898ff2a2dd01fe. Overflow checks are also enabled globally.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I2 Missing Event Emission

INFO FIXED

Description

We found that the following instructions are marked with `#[event_cpi]` but don't emit any events:

- futarchy/admin_approve_execute_multisig_proposal
- futarchy/execute_spending_limit_change
- price_based_performance_package/burn_performance_package

Furthermore, the following two instructions use `emit!` instead of `emit_cpi!`:

- price_based_performance_package/change_performance_package_authority
- price_based_performance_package/execute_change

Mitigation Suggestion

Emit events in all instructions using the same pattern for consistency.

Remediation

`emit!` has been updeted to `emit_cpi!` in this commit: https://github.com/metaDAOproject/programs/commit/7a93dff3e3cfe71fbf9f3346671fb5f47dcf0419, and the missing events were acknowledged.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I3 Refund Requires User ATA

INFO FIXED

Description

We found that the launchpad refund mechanism requires that the funder's token account is their ATA, while on deposit this wasn't required. If a funder therefore funds a launch from a wallet whose ATA has been reassigned to another authority, it will be possible for them to fund a launch, but a refund will not work, locking their tokens in the program.

Location

https://github.com/metaDAOproject/programs/blob/d05c432f9b67221267b0b3fd009cfd0227b4605b/programs/v07_launchpad/src/instructions/refund.rs#L35-L36

Relevant Code

```
/// refund.rs L35-L36
    #[account(mut, associated_token::mint = launch.quote_mint, associated_token::authority = funder)]
    pub funder_quote_account: Account<'info, TokenAccount>,
```

Mitigation Suggestion

Lift the ATA requirement for refunds. The token account should be any token account belonging to the funder and quote mint.

Remediation

The deposit has been restricted to ATA as well:
https://github.com/metaDAOproject/programs/commit/7d7a65dc230d2035d7f1bdb50f453c528b3b18d1

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I4 `resize_dao` Uses DAO Balance for Rent Calculation and Can Misbehave if DAO Holds SOL

INFO | UNKNOWN

Description

We found that `resize_dao.rs` uses `dao.lamports` to calculate how many lamports are needed for the reallocation. This is currently not an issue if the DAO account only holds rent, but it becomes problematic if the DAO later holds additional SOL. In that case, the calculation can become incorrect and may underfund or overfund the resize operation.

Location

https://github.com/metaDAOproject/programs/blob/d05c432f9b67221267b0b3fd009cfd0227b4605b/programs/futarchy/src/instructions/resize_dao.rs#L66

Relevant Code

```
if lamports_needed > dao.lamports() {
```

Mitigation Suggestion

Compute the required lamports using only the rent difference between old and new sizes: `minimum_balance(AFTER) - minimum_balance(BEFORE)`.

Remediation

Acknowledged as this is a single use instruction.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I5 Performance Package Init Accepts Default Pubkey for Oracle, Grantee, and Authority

INFO | UNKNOWN

Description

When initializing a performance package, the caller can pass the default pubkey (`Pubkey::default()`, the system program address) as the oracle account, the grantee, or the performance package authority. There is no validation that these pubkeys are not the default. Doing so by accident (or misuse) leads to broken or unrecoverable behavior.

• **Oracle** (`params.oracle_config.oracle_account`): Stored in the package and used by `start_unlock` and `complete_unlock` via `#[account(address = performance_package.oracle_config.oracle_account)]`. If it is the default pubkey, the program will expect the system program account to hold oracle data at the configured byte offset, which it does not. This can cause incorrect price reads, failed CPI, or confusing failures when unlocking.

• **Grantee** (`params.grantee`): Stored as `recipient` on the performance package. Unlocked tokens are sent to this address. If it is the default pubkey, tokens would be sent to the system program and are effectively lost.

• **Authority** (`params.performance_package_authority`): Stored on the package and required to sign for `change_performance_package_authority` and to approve changes in `execute_change`. If it is the default pubkey, no one can sign as the system program, so oracle or other configuration changes become impossible and control is effectively locked.

The params struct and initialization do not reject these values:

```
pub struct InitializePerformancePackageParams {
    pub tranches: Vec<Tranche>,
    pub min_unlock_timestamp: i64,
    pub oracle_config: OracleConfig,  // contains oracle_account: Pubkey
    pub twap_length_seconds: u32,
    pub grantee: Pubkey,
    pub performance_package_authority: Pubkey,
}
// ... in handle(): no require_neq!(..., Pubkey::default()) for any of these
```

Location

/programs/price_based_performance_package/src/instructions/initialize_performance_package.rs#L7-L15
/programs/price_based_performance_package/src/instructions/initialize_performance_package.rs#L58-L82
/programs/price_based_performance_package/src/state/performance_package.rs#L21-L24

Relevant Code

```
// InitializePerformancePackageParams – no check that pubkeys are not default
pub struct InitializePerformancePackageParams {
    pub tranches: Vec<Tranche>,
    pub min_unlock_timestamp: i64,
    pub oracle_config: OracleConfig,
    pub twap_length_seconds: u32,
    pub grantee: Pubkey,
    pub performance_package_authority: Pubkey,
}

pub struct OracleConfig {
    pub oracle_account: Pubkey,
    pub byte_offset: u32,
}

// initialize_performance_package handle() – values stored without validation
performance_package.set_inner(PerformancePackage {
    ...
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
    recipient: grantee,
    ...
    performance_package_authority,
    oracle_config,
    ...
});
```

Mitigation Suggestion

We recommend adding explicit checks at the start of `handle` to reject the default pubkey for oracle, grantee, and authority:

```
require_neq!(
    params.oracle_config.oracle_account,
    Pubkey::default(),
    PriceBasedPerformancePackageError::InvalidOracleAccount
);
require_neq!(
    params.grantee,
    Pubkey::default(),
    PriceBasedPerformancePackageError::InvalidGrantee
);
require_neq!(
    params.performance_package_authority,
    Pubkey::default(),
    PriceBasedPerformancePackageError::InvalidAuthority
);
```

Remediation

Marked as won't fix.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I6 When a Proposal is Rejected by the Market The Squads Proposal Should Be Rejected

INFO FIXED

Description

We found that when a proposal is rejected by the market, the associated Squads proposal will be unaffected and remain open for voting (Active). Instead, the correct action would be to vote Reject on the proposal, which would make the Squads proposal Rejected.

Location

https://github.com/metaDAOproject/programs/blob/d05c432f9b67221267b0b3fd009cfd0227b4605b/programs/futarchy/src/instructions/finalize_proposal.rs#L181-L204

Relevant Code

```
/// finalize_proposal.rs L181-L204
        if new_proposal_state == ProposalState::Passed {
            squads_multisig_program::cpi::proposal_approve(
                CpiContext::new_with_signer(
                    squads_multisig_program.to_account_info(),
                    squads_multisig_program::cpi::accounts::ProposalVote {
                        proposal: squads_proposal.to_account_info(),
                        multisig: squads_multisig.to_account_info(),
                        member: dao.to_account_info(),
                    },
                    dao_signer,
                ),
                squads_multisig_program::ProposalVoteArgs { memo: None },
            )?;

            spot.base_reserves += pass.base_reserves;
            spot.quote_reserves += pass.quote_reserves;
            spot.base_protocol_fee_balance += pass.base_protocol_fee_balance;
            spot.quote_protocol_fee_balance += pass.quote_protocol_fee_balance;
        } else {
            spot.base_reserves += fail.base_reserves;
            spot.quote_reserves += fail.quote_reserves;
            spot.base_protocol_fee_balance += fail.base_protocol_fee_balance;
            spot.quote_protocol_fee_balance += fail.quote_protocol_fee_balance;
        }
```

Mitigation Suggestion

Call the squads program in both cases, voting reject if the proposal was rejected by the market.

Remediation

Fixed in https://github.com/metaDAOproject/programs/commit/e8c014ef40c364eb085745b11d328d98551caf78.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I7 Invariants in DAO State Can Be Improved

INFO | ACKNOWLEDGED

Description

The `Dao::invariant()` function in `programs/futarchy/src/state/dao.rs` contains some validation checks for critical DAO configuration parameters. Currently, only 5 basic checks are performed:

1. Proposal duration ≥ 2 × TWAP start delay

2. Proposal duration ≥ 1 day (86,400 seconds)

3. Pass threshold ≤ 1000 bps (10%)

4. Team sponsored pass threshold ≥ -1000 bps (-10%)

5. Team sponsored pass threshold ≤ 1000 bps (10%)

However, many other critical parameters are not validated, which could lead to:

• **Economic attacks**: Invalid liquidity requirements or staking amounts could enable spam attacks or make the protocol unusable

• **TWAP manipulation**: Missing validation of TWAP parameters could allow for oracle manipulation

• **System instability**: Zero or excessively large values could cause arithmetic errors, division by zero, or overflow conditions

• **Governance bypass**: Invalid threshold configurations could allow proposals to pass/fail inappropriately

Since the `invariant()` function is called in both `initialize_dao()` (line 219) and `update_dao()` (line 76), missing validations affect both DAO creation and updates, allowing malicious or misconfigured parameters to persist.

Location

https://github.com/metaDAOproject/programs/blob/d05c432f9b67221267b0b3fd009cfd0227b4605b/programs/futarchy/src/state/dao.rs#L79

Relevant Code

```
    pub fn invariant(&self) -> Result<()> {
        require_gte!(
            self.seconds_per_proposal,
            self.twap_start_delay_seconds * 2,
            FutarchyError::ProposalDurationTooShort
        );

        require_gte!(
            self.seconds_per_proposal,
            60 * 60 * 24,
            FutarchyError::ProposalDurationTooShort
        );

        require_gte!(
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
            1_000,
            self.pass_threshold_bps,
            FutarchyError::PassThresholdTooHigh
        );

        require_gte!(
            self.team_sponsored_pass_threshold_bps,
            -1_000,
            FutarchyError::InvalidTeamSponsoredPassThreshold
        );

        require_gte!(
            1_000,
            self.team_sponsored_pass_threshold_bps,
            FutarchyError::InvalidTeamSponsoredPassThreshold
        );

        Ok(())
    }
```

Mitigation Suggestion

Add the following invariant checks to prevent invalid DAO configurations:

1. TWAP Oracle Parameters:

```
// Prevent TWAP manipulation through excessively large observation changes
require_neq!(
    self.twap_initial_observation,
    0,
    FutarchyError::InvalidInitialObservation
);

require_neq!(
    self.twap_max_observation_change_per_update,
    0,
    FutarchyError::InvalidMaxObservationChange
);

// Ensure TWAP delay is reasonable (between 1 minute and proposal duration)
require_gte!(
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
    self.twap_start_delay_seconds,
    60,
    FutarchyError::InvalidStartDelaySlots
);

require_gte!(
    self.seconds_per_proposal,
    self.twap_start_delay_seconds,
    FutarchyError::InvalidStartDelaySlots
);

// Prevent observation change from being larger than initial observation by orders of magnitude
// This prevents manipulation where observation can swing wildly
require_gte!(
    self.twap_initial_observation,
    self.twap_max_observation_change_per_update,
    FutarchyError::InvalidMaxObservationChange
);
```

2. Liquidity Requirements:

```
// Ensure minimum liquidity requirements are set to prevent spam
require_neq!(
    self.min_quote_futarchic_liquidity,
    0,
    FutarchyError::InsufficientLiquidity
);

require_neq!(
    self.min_base_futarchic_liquidity,
    0,
    FutarchyError::InsufficientLiquidity
);

// Ensure liquidity requirements are reasonable relative to each other
// to prevent extreme imbalances
```

3. Staking Requirements:

```
// Ensure staking is required to prevent free proposal creation
require_neq!(
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
    self.base_to_stake,
    0,
    FutarchyError::InsufficientStakeToLaunch
);
```

4. Pubkey Validity:

```
// Ensure critical addresses are not default/system program
require_keys_neq!(
    self.squads_multisig,
    Pubkey::default(),
    FutarchyError::InvalidAdmin
);

require_keys_neq!(
    self.squads_multisig_vault,
    Pubkey::default(),
    FutarchyError::InvalidAdmin
);

require_keys_neq!(
    self.team_address,
    Pubkey::default(),
    FutarchyError::InvalidAdmin
);

require_keys_neq!(
    self.base_mint,
    self.quote_mint,
    FutarchyError::InvalidAmount
);
```

5. Spending Limit Validation:

```
// If spending limit is set, validate member count
if let Some(ref limit) = self.initial_spending_limit {
    require_gte!(
        MAX_SPENDING_LIMIT_MEMBERS,
        limit.members.len(),
        FutarchyError::InvalidAmount
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
    );

    // Ensure amount is not zero
    require_neq!(
        limit.amount_per_month,
        0,
        FutarchyError::InvalidAmount
    );
}
```

6. Proposal Duration Upper Bound:

```
// Prevent excessively long proposals (e.g., max 1 year)
require_gte!(
    365 * 24 * 60 * 60, // 1 year in seconds
    self.seconds_per_proposal,
    FutarchyError::ProposalDurationTooShort // Reuse or create new error
);
```

These additional invariants will ensure that:

- TWAP oracles cannot be manipulated through invalid initial states
- Economic parameters prevent spam and ensure meaningful proposals
- Critical addresses are properly configured
- The system cannot enter states that would cause arithmetic errors or undefined behavior

Remediation

Acknowledged.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I8 Misleading Variable Name `input_amount_with_lp_fee` in Futarchy AMM Swap

INFO FIXED

Description

In the futarchy AMM swap logic, the variable `input_amount_with_lp_fee` is misnamed. It holds the input amount **after** the LP taker fee is applied (i.e. the amount that participates in the curve), not "with" the fee. The computation multiplies by `(MAX_BPS - LP_TAKER_FEE_BPS)`, so the fee is effectively deducted. The name suggests the opposite and should be updated for clarity.

```
let input_amount_with_lp_fee =
    input_amount_after_protocol_fee as u128 * (MAX_BPS - LP_TAKER_FEE_BPS) as u128;
```

Location

/programs/futarchy/src/state/futarchy_amm.rs#L512-L513

Relevant Code

```
// futarchy_amm.rs - misnamed: value is input amount after LP fee is deducted
let input_amount_with_lp_fee =
    input_amount_after_protocol_fee as u128 * (MAX_BPS - LP_TAKER_FEE_BPS) as u128;
```

Mitigation Suggestion

Rename the variable to reflect that it is the input amount after (or net of) the LP fee, for example:

```
let input_amount_after_lp_fee =
    input_amount_after_protocol_fee as u128 * (MAX_BPS - LP_TAKER_FEE_BPS) as u128;
```

Update all usages of `input_amount_with_lp_fee` in the same function to `input_amount_after_lp_fee`.

Remediation

Fixed in https://github.com/metaDAOproject/programs/commit/5ae14194d697647d08e484f46f23dc0484963c1f

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I9 Outdated "Autocrat" References Should Be Replaced With "Futarchy"

INFO FIXED

Description

The futarchy program was previously named "autocrat" but has been renamed. Multiple references to "autocrat" remain in comments and variable names throughout the codebase, creating confusion and inconsistency. These should be updated to "futarchy" to reflect the current program name.

Location

/programs/v07_launchpad/src/instructions/complete_launch.rs#L38-L39
/programs/v07_launchpad/src/instructions/complete_launch.rs#L181
/programs/v07_launchpad/src/instructions/complete_launch.rs#L185
/programs/v07_launchpad/src/instructions/complete_launch.rs#L189
/programs/v07_launchpad/src/instructions/complete_launch.rs#L193
/programs/v07_launchpad/src/instructions/complete_launch.rs#L197
/programs/v07_launchpad/src/instructions/complete_launch.rs#L400
/programs/v07_launchpad/src/instructions/complete_launch.rs#L509
/programs/futarchy/src/instructions/collect_meteora_damm_fees.rs#L40
/programs/futarchy/src/instructions/admin_approve_execute_multisig_proposal.rs#L18
/programs/v07_launchpad/src/instructions/initialize_performance_package.rs#L42

Relevant Code

```
// complete_launch.rs - StaticCompleteLaunchAccounts struct
pub struct StaticCompleteLaunchAccounts<'info> {
    pub futarchy_program: Program<'info, Futarchy>,
    pub token_metadata_program: Program<'info, Metadata>,
    /// CHECK: checked by autocrat program
    pub autocrat_event_authority: UncheckedAccount<'info>,
    // ...
}

// complete_launch.rs - CompleteLaunch struct
/// CHECK: init by autocrat
#[account(mut, seeds = [b"amm_position", dao.key().as_ref(), squads_multisig_vault.key().as_ref()], bump, see
ds::program = static_accounts.futarchy_program)]
pub dao_owned_lp_position: UncheckedAccount<'info>,

/// CHECK: checked by autocrat
#[account(mut)]
pub futarchy_amm_base_vault: UncheckedAccount<'info>,

/// CHECK: checked by autocrat
#[account(mut)]
pub futarchy_amm_quote_vault: UncheckedAccount<'info>,

/// CHECK: this is the DAO account, init by autocrat
#[account(mut)]
pub dao: UncheckedAccount<'info>,

/// CHECK: checked by autocrat program
#[account(mut, seeds = [squads_multisig_program::SEED_PREFIX, squads_multisig_program::SEED_MULTISIG, dao.key
().as_ref()], bump, seeds::program = static_accounts.squads_program)]
pub squads_multisig: UncheckedAccount<'info>,

// complete_launch.rs - Usage in initialize_dao
event_authority: self
    .static_accounts
    .autocrat_event_authority
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
    .to_account_info(),

// complete_launch.rs - Usage in provide_futarchy_amm_liquidity
event_authority: self
    .static_accounts
    .autocrat_event_authority
    .to_account_info(),

// collect_meteora_damm_fees.rs
/// CHECK: checked by autocrat program
#[account(mut, seeds = [squads_multisig_program::SEED_PREFIX, squads_multisig_program::SEED_MULTISIG, dao.key
().as_ref()], bump, seeds::program = squads_program)]
pub squads_multisig: Account<'info, squads_multisig_program::Multisig>,

// admin_approve_execute_multisig_proposal.rs
/// CHECK: checked by autocrat program
#[account(mut, seeds = [squads_multisig_program::SEED_PREFIX, squads_multisig_program::SEED_MULTISIG, dao.key
().as_ref()], bump, seeds::program = squads_multisig_program)]
pub squads_multisig: Account<'info, squads_multisig_program::Multisig>,

// initialize_performance_package.rs
/// CHECK: this is the DAO account, init by autocrat
#[account(address = launch.dao.as_ref().unwrap().key())]
pub dao: UncheckedAccount<'info>,
```

Mitigation Suggestion

We recommend you to perform a systematic find-and-replace across the codebase:

1. Replace all comment references from `autocrat program` to `futarchy program` 2. Rename the field `autocrat_event_authority` to `futarchy_event_authority` in `StaticCompleteLaunchAccounts` 3. Update all usages of `autocrat_event_authority` to `futarchy_event_authority` 4. Update comments from `init by autocrat` to `init by futarchy program` 5. Update comments from `checked by autocrat` to `checked by futarchy program`

This ensures consistency with the current program name and reduces confusion for developers and auditors.

Remediation

Fixed in https://github.com/metaDAOproject/programs/commit/a1beb2fc87946a799716cf63d6026f8f32164069

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I10 Incorrect Reference Link in Sqrt Price Calculation

INFO FIXED

Description

The `sqrt_price` calculation in `provide_single_sided_meteora_liquidity` references the wrong lines in the Meteora DAMM v2 SDK. The code comment points to lines 121-133, but the actual reference implementation for `getSqrtPriceFromPrice` is located at lines 135-152.

```
// ref: https://github.com/MeteoraAg/damm-v2-sdk/blob/3d740ea8434af20a024d5d6fd08d60792dca9ca4/src/helpers/utils.ts#L121-L133
let float_price = final_raise_amount as f64 / TOKENS_TO_PARTICIPANTS as f64;
let sqrt_price = (float_price.sqrt() * 2_f64.powf(64.0)) as u128;
```

Location

/programs/v07_launchpad/src/instructions/complete_launch.rs#L581-L583

Relevant Code

```
// complete_launch.rs - incorrect reference link
// ref: https://github.com/MeteoraAg/damm-v2-sdk/blob/3d740ea8434af20a024d5d6fd08d60792dca9ca4/src/helpers/utils.ts#L121-L133
let float_price = final_raise_amount as f64 / TOKENS_TO_PARTICIPANTS as f64;
let sqrt_price = (float_price.sqrt() * 2_f64.powf(64.0)) as u128;
```

Mitigation Suggestion

We recommend you to update the reference link to point to the correct lines:

```
// ref: https://github.com/MeteoraAg/damm-v2-sdk/blob/3d740ea8434af20a024d5d6fd08d60792dca9ca4/src/helpers/utils.ts#L135-L152
let float_price = final_raise_amount as f64 / TOKENS_TO_PARTICIPANTS as f64;
let sqrt_price = (float_price.sqrt() * 2_f64.powf(64.0)) as u128;
```

Remediation

Fixed in https://github.com/metaDAOproject/programs/commit/f72bdba4b4364a2d1570627f8ab844eb3df9a602

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I11 Duplicate Optimistic Proposal Check in `initiate_vault_spend_optimistic_proposal`

INFO FIXED

Description

The `initiate_vault_spend_optimistic_proposal` instruction contains duplicate code that performs the same validation check twice with different error messages. Both checks validate that `dao.optimistic_proposal.is_none()`, but use different error types, creating confusion and potential maintenance issues.

```
// There should be no active optimistic proposal
require!(
    self.dao.optimistic_proposal.is_none(),
    FutarchyError::ActiveOptimisticProposalAlreadyEnqueued
);

// No existing optimistic proposal can be present. If one has passed, it can be finalized (finalize_optimisti
c_proposal)
require!(
    self.dao.optimistic_proposal.is_none(),
    FutarchyError::ProposalDurationTooShort
);
```

The same condition is checked twice consecutively (lines 69-72 and 75-78), but with different error messages. The second check uses `FutarchyError::ProposalDurationTooShort`, which is misleading since it has nothing to do with proposal duration. This duplicate code adds unnecessary overhead, creates confusion about which error would actually be returned (the first one), and makes the code harder to maintain.

Location

/programs/futarchy/src/instructions/initiate_vault_spend_optimistic_proposal.rs#L68-L78

Relevant Code

```
// initiate_vault_spend_optimistic_proposal.rs - duplicate check with correct error
require!(
    self.dao.optimistic_proposal.is_none(),
    FutarchyError::ActiveOptimisticProposalAlreadyEnqueued
);

// initiate_vault_spend_optimistic_proposal.rs - duplicate check with incorrect error message
require!(
    self.dao.optimistic_proposal.is_none(),
    FutarchyError::ProposalDurationTooShort // Wrong error type
);
```

Mitigation Suggestion

We recommend you to remove the duplicate check (lines 74-78) and keep only the first validation with the correct error message:

```
// There should be no active optimistic proposal
require!(
    self.dao.optimistic_proposal.is_none(),
    FutarchyError::ActiveOptimisticProposalAlreadyEnqueued
);
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

Remediation

Fixed in https://github.com/metaDAOproject/programs/commit/98dfe15ae1d0c89ac494659f2029f18072f0f373

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I12 `unreachable!()` in AMM Operations leads to no error returns

INFO FIXED

Description

The `provide_liquidity` and `withdraw_liquidity` instructions use `unreachable!()` when the pool is not in Spot state, causing the program to panic instead of returning a proper error. This can be triggered when these instructions are called while a proposal is Pending and the pool is in Futarchy state.

```
let PoolState::Spot { ref mut spot } = dao.amm.state else {
    // TODO: check that pool is already in right state
    unreachable!();
};
```

When a user calls `provide_liquidity` or `withdraw_liquidity` while a proposal is Pending (pool is in Futarchy state), the code hits `unreachable!()` and panics. While panicking correctly reverts, using `unreachable!()` instead of a proper error return makes the program less robust and can cause unexpected failures in integrated frontend or bot systems that may not handle panics gracefully.

The `spot_swap` instruction handles this scenario correctly by calling `dao.amm.state.swap()` which returns a proper error when the pool state doesn't match the market type, demonstrating the correct approach.We found...

Location

/programs/futarchy/src/instructions/provide_liquidity.rs#L86-L89
/programs/futarchy/src/instructions/withdraw_liquidity.rs#L95-L98
/programs/futarchy/src/instructions/withdraw_liquidity.rs#L122-L124

Relevant Code

```
// provide_liquidity.rs - panics on non-Spot state
let PoolState::Spot { ref mut spot } = dao.amm.state else {
    // TODO: check that pool is already in right state
    unreachable!();
};

// withdraw_liquidity.rs - panics on non-Spot state (first occurrence)
let (base_to_withdraw, quote_to_withdraw) = {
    let PoolState::Spot { ref spot } = dao.amm.state else {
        // TODO: check that pool is already in right state
        unreachable!();
    };
    spot.get_base_and_quote_withdrawable(
        liquidity_to_withdraw as u64,
        total_liquidity as u64,
    )
};

// withdraw_liquidity.rs - panics on non-Spot state (second occurrence)
{
    let PoolState::Spot { ref mut spot } = dao.amm.state else {
        unreachable!();
    };
    spot.base_reserves -= base_to_withdraw;
    spot.quote_reserves -= quote_to_withdraw;
}
```

Mitigation Suggestion

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

We recommend you to replace `unreachable!()` with a proper error return using the existing `FutarchyError::PoolNotInSpotState` error:

```
let PoolState::Spot { ref mut spot } = dao.amm.state else {
    return Err(FutarchyError::PoolNotInSpotState.into());
};
```

Remediation

Fixed in https://github.com/metaDAOproject/programs/commit/c9db7a945f9578abded5ee1143e884e97a912225

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

APPENDIX

Vulnerability Classification

We rate our issues according to the following scale. Informational issues are reported informally to the developers and are not included in this report.

Severity	Description
Critical	Vulnerabilities that can be easily exploited and result in loss of user funds, or directly violate the protocol's integrity. Immediate action is required.
High	Vulnerabilities that can lead to loss of user funds under non-trivial preconditions, loss of fees, or permanent denial of service that requires a program upgrade. These issues require attention and should be resolved in the short term.
Medium	Vulnerabilities that may be more difficult to exploit but could still lead to some compromise of the system's functionality. For example, partial denial of service attacks, or such attacks that do not require a program upgrade to resolve, but may require manual intervention. These issues should be addressed as part of the normal development cycle.
Low	Vulnerabilities that have a minimal impact on the system's operations and can be fixed over time. These issues may include inconsistencies in state, or require such high capital investments that they are not exploitable profitably.
Informational	Findings that do not pose an immediate risk but could affect the system's efficiency, maintainability, or best practices.

Audit Methodology

Accretion is a boutique security auditor specializing in Solana's ecosystem. We employ a customized approach for each client, strategically allocating our resources to maximize code review effectiveness. Our auditors dedicate substantial time to developing a comprehensive understanding of each program under review, examining design decisions, expected and edge-case behaviors, invariants, optimizations, and data structures, while meticulously verifying mathematical correctness—all within the context of the developers' intentions.

Our audit scope extends beyond on-chain components to include associated infrastructure, such as user interfaces and supporting systems. Every audit encompasses both a holistic protocol design review and detailed line-by-line code analysis.

During our assessment, we focus on identifying:
• Solana-specific vulnerabilities
• Access control issues
• Arithmetic errors and precision loss
• Race conditions and MEV opportunities
• Logic errors and edge cases
• Performance optimization opportunities
• Invariant violations
• Account confusion vulnerabilities
• Authority check omissions
• Token22 implementation risks and SPL-related pitfalls
• Deviations from best practices

Our approach transcends conventional vulnerability classifications. We continuously conduct ecosystem-wide security research to identify and mitigate emerging threat vectors, ensuring our audits remain at the forefront of Solana security practices.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.